IT271 ENTERPRISE DATA WAREHOUSING · SEMESTER PROJECT

Disaster Response Intelligence Warehouse (DRIW)

Design, implementation and validation of an enterprise data warehouse for disaster impact, evacuation, relief and emergency-response analytics in the Philippines, 2020–2025

Author: Ivan Christian Salinas

Course: IT271 Enterprise Data Warehousing

Submitted: October 16, 2026

Repository: github.com/sardinexszc/NEUSTIT271_EDW_Project

Data notice: event names, dates and PSGC geography are real. All damage, evacuation, relief and response figures are synthetic, generated for demonstration.

Contents

Executive Summary 4

What is real and what is simulated 4

Results 4

How to read this report 4

Requirements Coverage 5

1. Business Case and Requirements Analysis 6

1.1 Enterprise domain 6

1.2 Stakeholders and BI objectives 6

1.3 Key business questions 6

1.4 Functional requirements 7

1.5 Non-functional requirements 7

1.6 Assumptions and constraints 8

2. Data Warehouse Design 9

2.1 Conceptual model 9

2.2 Logical model 9

2.3 Physical model 10

2.4 DRIW Dimensional Model 13

3. Data Dictionary and Metadata 16

3.1 Schemas (layers) 16

3.2 Lineage 16

3.3 dw · Star schema 17

3.4 mart · Data marts 21

3.5 Supporting layers 23

4. ETL Design 26

4.1 Overview 26

4.2 Data sources 26

4.3 Extract (raw layer) 27

4.4 Transform (staging and data-quality layers, dbt) 27

4.5 Load (warehouse and marts) 27

4.6 Orchestration, scheduling and error handling 28

4.7 Tools 28

5. Data Quality Framework 29

5.1 Where data quality happens 29

5.2 Severity policy 29

5.3 Key techniques 29

5.4 Proof: reconciliation against planted defects 30
5.5 Running 30
6. Data Governance, Security and Privacy 31
6.1 Roles 31
6.2 Row-level security (regional users) 31
6.3 Surviving rebuilds 32
6.4 Auditing access 32
6.5 Privacy 32
6.6 Supabase 33
6.7 Standards alignment 33
7. Analytics: OLAP and Dashboards 34
7.1 OLAP approach (ROLAP) 34
7.2 Dashboards 34
7.3 Sample findings (synthetic data) 39
8. Testing and Validation 40
8.1 Summary 40
8.2 What is validated 40
8.3 Defects found by testing (and fixed) 41
8.4 Known limitations 41
8.5 Query Performance Test Report 41
9. Implementation and Live Demo 44
9.1 Technology stack 44
9.2 Repository layout 44
9.3 Build and run 44
9.4 Live demo script 44
Appendix A. Sample Analytical Queries 46

Executive Summary

Problem. A Philippine typhoon or monsoon flood is reported by several agencies at once: NDRRMC (damage), DSWD (evacuation centers), NGOs and UN agencies (relief), PAGASA (the hazard) and LGUs (response requests). Their systems share no place codes, event names or units, so leadership cannot answer cross-agency questions such as "which provinces got the least relief per affected person?" without weeks of manual reconciliation.

Solution. DRIW, the Disaster Response Intelligence Warehouse, integrates the five sources into one dimensional model on PostgreSQL 17: four star schemas (transaction, periodic-snapshot and accumulating-snapshot facts) over nine conformed dimensions, including a Type 2 slowly changing location dimension. A Python and dbt ETL pipeline cleans and conforms the data with 36 data-quality rules; six ROLAP marts (`GROUP BY CUBE` and `GROUPING SETS`) feed Executive, Operations and Logistics dashboards. A PHP back-end on XAMPP serves them live through a read-only login, and row-level security limits regional users to their own region.

What is real and what is simulated

Real	Simulated (synthetic)
The architecture, ETL pipeline, data-quality rules, star schema, marts, security and dashboards: built and run exactly as they would be on agency data	Every damage, casualty, evacuation, relief and response **figure** in this report and on the dashboards
The 31 events (PAGASA names and dates, 2020–2025)	The source files themselves (CSV, Excel, JSON) and their data-entry defects
All geography: PSA PSGC, 17 regions, 81 provinces, 1,634 cities and municipalities, 42,046 barangays	

Agency operational data is not public, so the figures are generated (seed 271), scaled to population and storm strength. **The numbers demonstrate the warehouse; they are not findings about real disasters.** Synthetic data has one advantage: 9,995 defects of 19 known types were planted on purpose, so the ETL can be proven correct by counting. Nothing in the pipeline depends on the data being synthetic: replacing the generator with real agency extracts changes the input files only (see the Privacy section of Chapter 6).

Results

Measure	Result
Source rows integrated	85,841 from 5 systems in 4 formats
Planted defects detected	19 of 19 types, each at the exact count (9,995 defects)
Location names matched to PSGC	63,347 rows, 0 false matches
Automated checks	99 dbt checks and 62 end-to-end, governance and API tests, all passing
Query performance	14 of 14 sample queries within 1 s (slowest about 32 ms); marts about 78× faster than the facts
Access control	4 roles; row-level security verified per role

How to read this report

The Requirements Coverage page that follows maps each item of the project brief to the chapter that meets it. Chapters 1–2 cover the business case and the design, 3–5 the metadata, ETL and data quality, 6 governance and privacy, 7 the analytics, 8 testing, and 9 how to run the live demo. Appendix A lists the sample queries.

Requirements Coverage

Each requirement of the IT271 semester project brief, and where this submission meets it.

Brief section	Requirement	Where it is met
1. Overview	Requirements gathering and BI objectives	Chapter 1 (Business Case), §2–§5
1. Overview	Data modeling: conceptual, logical, physical	Chapter 2 (Warehouse Design), §1–§3
1. Overview	ETL processes	Chapter 4 (ETL Design)
1. Overview	OLAP cube or star/snowflake schema	Chapter 2 (four star schemas); Chapter 7 (ROLAP cube)
1. Overview	Data governance and security	Chapter 6 (Governance, Security and Privacy)
1. Overview	Analytics, dashboards, reports	Chapter 7 (Analytics)
2A	Description of the enterprise domain	Chapter 1 §1
2A	Key business questions	Chapter 1 §3 (BQ1–BQ10)
2A	Functional and non-functional requirements	Chapter 1 §4 (FR1–FR12), §5 (NFR1–NFR9)
2A	Schema design: star or snowflake (fact + dimension tables)	Chapter 2 §2; Chapter 2 (dimensional model detail)
2A	ER/UML diagrams of the warehouse design	Chapter 2: conceptual diagram and four ER diagrams
2A	Data dictionary and metadata documentation	Chapter 3 (generated from the catalog; lineage, row counts, load times)
2A	ETL: sources of data (mock or real)	Chapter 4 §2
2A	ETL: transformation logic (cleaning, integration, aggregation)	Chapter 4 §4; Chapter 5
2A	ETL workflow diagram	Chapter 4 §1
2A	Governance: security, privacy, access controls	Chapter 6 (roles, row-level security, web login, Privacy section)
2A	Governance: data quality management strategy	Chapter 5 (Data Quality Framework)
2B	Warehouse on SQL Server, Oracle, PostgreSQL, MySQL or cloud DW	PostgreSQL 17 (Chapter 2 §3, Chapter 9)
2B	ETL pipelines (tools or custom Python/SQL)	Python + dbt, `ingestion/pipeline.py` (Chapter 4)
2B	OLAP cube / analytical queries	`mart_impact_cube` (CUBE), GROUPING SETS, 14 sample queries (Chapter 7, Appendix A)
2B	Reports/dashboards (Power BI, Tableau or similar)	Executive, Operations, Logistics dashboards on XAMPP (Chapter 7 §2)
2C	Validation of ETL: completeness, correctness, accuracy	Chapter 8 (99 dbt checks, 28 end-to-end, 19/19 reconciliation, 0 false matches)
2C	Test reports for data consistency and query performance	Chapter 8 (consistency); Query Performance Test Report
2D	Live demo	Chapter 9 §4 (demo script)
2D	Showcase of sample queries, OLAP analysis, dashboards	Appendix A, Chapter 7
2D	Defense of design choices	Chapter 2 (star vs snowflake, ROLAP), dimensional-model design decisions, Chapter 4 (full reload, quarantine)

1. Business Case and Requirements Analysis

1.1 Enterprise domain

Government: disaster risk reduction and management (DRRM) in the Philippines.

The Philippines is struck by around twenty tropical cyclones a year, plus monsoon floods. Responding to one event involves many agencies, each with its own system:

Organization	Role in a disaster	System of record (simulated)
NDRRMC / Office of Civil Defense	Consolidates damage and casualty figures	Situation reports (CSV), cumulative, re-issued per event
DSWD (DROMIC)	Runs and reports evacuation centers	Daily Excel workbooks per field office
NGOs and UN agencies (PRC, Caritas, WFP, UNICEF) and DSWD	Distribute relief goods	Partner reports (nested JSON) with their own item codes and units
PAGASA	Names and tracks hazard events	Weather bulletins (CSV)
LGUs / MDRRMOs	Rescue, evacuation, road clearing, relief delivery	Response request logs (CSV)

These systems share no place codes, event names or units. Leadership therefore cannot answer cross-agency questions without manual reconciliation in spreadsheets.

Scope. Tropical cyclones and southwest-monsoon floods, 2020–2025: 31 events. Event names and dates (PAGASA) and all geography (PSA PSGC: 17 regions, 81 provinces, 1,634 cities and municipalities, 42,046 barangays) are real. Operational figures are synthetic, generated with counted, planted defects (85,841 source rows; 9,995 defects of 19 types), because agency data is not public and known defects let the ETL be proven correct.

1.2 Stakeholders and BI objectives

Stakeholder	Needs	BI objective	Served by
Executive leadership (NDRRMC council, agency heads)	Overall impact, trends, where to invest	Single view of damage, casualties and relief across all agencies and years	Executive dashboard, `mart_event_summary`, `mart_impact_cube`
Operations managers (regional DRRM offices)	During and after an event: evacuation load, response speed	Per-event evacuation and response performance	Operations dashboard, `secure.*` views for their region
Logistics officers	Whether relief matched need	Relief by organization and resource, coverage per province	Logistics dashboard, `mart_province_event_impact`
Data analysts	Ad-hoc analysis, forecasting inputs	Clean, conformed star schema with history	`dw.*`, sample queries
Data stewards / IT	Data quality, reconciliation, operations	Measurable data quality with traceability to source	`dq.*`, quarantine, lineage

1.3 Key business questions

#	Business question	Answered by
BQ1	Which events caused the most damage, deaths and displacement?	Q01, Q13; Executive dashboard
BQ2	How does damage trend by year and category, and how did each year compare with the last?	Q02, Q05, Q12; Executive dashboard
BQ3	Within an event, which regions, provinces and towns were hit hardest?	Q03 (drill-down)
BQ4	Which regions suffer most from typhoons versus monsoon floods?	Q04, Q05 (cube); Executive dashboard

BQ5	How many people were in evacuation centers at the peak, and for how long?	Q06; Operations dashboard
BQ6	Which evacuation centers are repeatedly overcrowded?	Q07; Operations dashboard
BQ7	How quickly are response teams dispatched, by request type and priority, and where is it slowest?	Q08, Q09; Operations dashboard
BQ8	Did relief reach provinces in proportion to impact? Which were under-served?	Q10; Logistics dashboard
BQ9	Which organizations delivered which kinds of relief?	Q11; Logistics dashboard
BQ10	How reliable is the data: what was rejected, and why?	Q14; `dq.dq_reconciliation`

The Q-numbers refer to `sample-queries.sql`.

1.4 Functional requirements

#	Requirement	How it is met	Status
FR1	Ingest data from the five agency systems in their native formats (CSV, Excel, JSON).	`ingestion/load_raw.py` lands every file in `raw.*` unchanged.	Met
FR2	Keep lineage from every warehouse row to its source file and row.	`meta.load_batch`, `_batch_id/_source_file/_row_number`, `dim_source_system`.	Met
FR3	Conform place names to the official PSGC codes.	`stg_location_resolution`: exact, normalized, fuzzy; 0 false matches on ground truth.	Met
FR4	Conform event names, organizations and relief units across sources.	Event aliases with time windows; organization and item alias maps; unit conversion.	Met
FR5	Detect, correct, flag or quarantine data-quality defects, with reasons.	36 rules in `dq_rules`; `dq_quarantine`; 19/19 planted defect types reconciled.	Met
FR6	Store integrated data in a dimensional model with conformed dimensions.	Star schema: 4 facts, 9 dimensions (`docs/warehouse-design.md`).	Met
FR7	Preserve history when administrative boundaries change.	SCD Type 2 `dim_location` (2022 Maguindanao split).	Met
FR8	Support OLAP analysis: roll-up, drill-down, slice, dice, pivot.	`GROUP BY CUBE / ROLLUP / GROUPING SETS`, marts, sample queries Q01–Q14.	Met
FR9	Provide dashboards for executive, operational and logistics users.	Next.js dashboards on XAMPP: Executive, Operations, Logistics.	Met
FR10	Let users filter by event, share views and export data.	`?event=` links, CSV download on every chart table.	Met
FR11	Restrict data by role, including region-level restriction.	4 roles, row-level security, `driw_web` least-privilege login.	Met
FR12	Rebuild the warehouse end to end with one command.	`python ingestion/pipeline.py`.	Met

1.5 Non-functional requirements

#	Requirement	Target	Evidence	Status
NFR 1	Data accuracy	Every planted defect type detected at the exact planted count	`dq.dq_reconciliation`: 19/19 match	Met
NFR 2	Query performance	Analytical queries under 1 second	`docs/query-performance.md`: 14/14, slowest about 32 ms	Met
NFR 3	Security	Least privilege; regional row-level restriction; no credentials in code	`docs/governance.md`; 21 per-role access tests + web-role test	Met
NFR 4	Privacy	No personal data in the warehouse; casualty data aggregated	`docs/governance.md` § Privacy	Met
NFR 5	Reliability	A failed source row never stops the load; rejected rows are kept with reasons	Quarantine design; staging keeps every row	Met
NFR 6	Reproducibility	Same inputs produce the same warehouse	Deterministic generator (seed 271); e2e test rebuilds from nothing	Met
NFR 7	Maintainability	Version-controlled, tested, documented	GitHub repo, dbt tests, CI on every push, generated data dictionary	Met
NFR 8	Usability and accessibility	Readable on desktop and phone; color-blind-safe charts; table view for every chart	Validated palette, legends, "Show table", responsive layout	Met
NFR 9	Portability	Run on the agency's own infrastructure without cloud services	Local PostgreSQL 17 + XAMPP (Apache, PHP); migrations also run on Supabase	Met

1.6 Assumptions and constraints

- Operational figures are synthetic; they are scaled to population and storm strength to be plausible, not to match official NDRRMC totals.
- City population is estimated from barangay counts, because the PSA workbook with census population could not be downloaded automatically.
- Loads are batch (full reload), not real time; near-real-time feeds are future work.
- Standards followed: UNDRR Global Framework for Disaster-Related Statistics, Sendai Framework 2015–2030, UNDRR Data Strategy and Roadmap 2023–2027, PSA PSGC, Data Privacy Act of 2012 (RA 10173).

2. Data Warehouse Design

The design moves from business concepts to tables in three levels. The logical level is detailed in dimensional-model.md (grains, bus matrix, SCD Type 2, design decisions); the full column list is in data-dictionary.md.

2.1 Conceptual model

The DRIW answers questions about four **business processes**, each recorded by a different agency. Every process involves the same core **entities**: a hazard event, a place, a date, the organizations involved and the source that reported it. These shared entities are what allow the processes to be analyzed together.

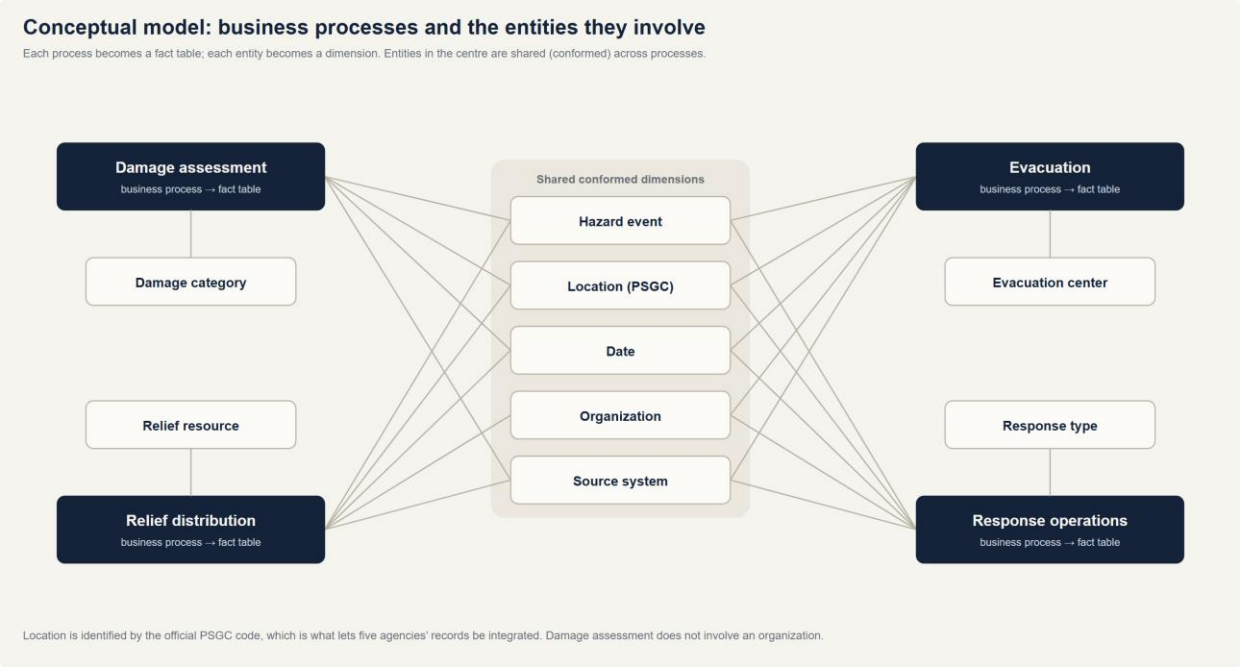


Figure: Conceptual model

Business process	Reported by	Key question it answers	Entities involved
Damage assessment	NDRRMC	How bad was it, and where?	Event, Location, Date, Damage category, Source
Evacuation	DSWD DROMIC	How many were displaced, and were centers adequate?	Event, Location, Date, Organization, Evacuation center, Source
Relief distribution	NGOs, UN agencies, DSWD	Did relief reach the people affected?	Event, Location, Date, Organization, Relief resource, Source
Response operations	LGUs / MDRRMOs	How fast did help arrive?	Event, Location, Date, Organization, Response type, Source

Why a dimensional (Kimball) design. The users ask aggregate, multi-perspective questions ("damage by region by year by hazard"), not transactional ones. A dimensional model makes those questions simple to write and fast to run, and conformed dimensions let the four processes be compared directly. The enterprise integration happens in the staging layer before data reaches the stars.

2.2 Logical model

Each business process becomes a **fact table** at a stated grain; each entity becomes a **dimension**. The model is a set of four **star schemas** sharing conformed dimensions (a "constellation").

Fact table	Type	Grain (one row per…)	Measures

fact_damage_assessment	Transaction	event × city/municipality × damage category	dead, injured, missing, affected families/persons, houses damaged, cost (PHP)
fact_evacuation_snapshot	Periodic snapshot	evacuation center × day × event	families inside, persons inside, capacity, over-capacity flag
fact_resource_distribution	Transaction	relief item in one distribution	quantity (standard unit), value (PHP), beneficiary families
fact_response_action	Accumulating snapshot	response request	hours to dispatch / arrive / on site / complete, personnel, completed flag

Dimension	Natural key	History	Hierarchy / notable attributes
dim_date	calendar date	Type 0	year → quarter → month → day; typhoon season; -1 "Not yet occurred"
dim_location	PSGC code	**Type 2**	region → province → city/municipality
dim_event	event code	Type 1	hazard type → category → event
dim_organization	org code	Type 1	org type → organization
dim_resource	item code	Type 1	category → item
dim_evacuation_center	EC code	Type 1	type, barangay, capacity
dim_damage_category	category	Type 0	—
dim_response_type	type code	Type 0	cluster → type
dim_source_system	batch id	—	lineage: source system → file

Star, not snowflake. Dimensions are kept denormalized (region and province names sit in dim_location rather than in separate region and province tables). This costs a little storage but keeps every query to one join per dimension, which suits the BI users, and the hierarchies are small. The PSGC master data itself stays normalized in ref.psgc, so the star is derived from a single authoritative source.

The bus matrix, measure additivity rules and the SCD Type 2 treatment of the 2022 Maguindanao split are in dimensional-model.md.

2.3 Physical model

Platform. PostgreSQL 17 (local; the same migrations run unchanged on Supabase). Database size about 80 MB.

Schemas as layers.

Schema	Contents	Built by	Materialization
raw	source data as received (text)	Python	tables, truncated and reloaded
ref, meta	master data, lineage, defect manifest	Python, dbt seeds	tables
staging	typed, conformed, quality-tagged data	dbt	tables
dq	rules, issue log, scorecard, quarantine, reconciliation	dbt	tables
dw	dimensions and facts	dbt	tables with PK/FK constraints and row-level security
mart	dashboard aggregates, CUBE, GROUPING SETS	dbt	materialized views
secure, governance	region-scoped views, role mapping	dbt, migration	views

Keys and data types.

- Surrogate keys are integers generated in dbt (row_number() over the natural key), so facts join on small integers. dim_date uses a smart key, YYYYMMDD.
- Natural keys are kept as attributes (psgc_code, event_code, ec_code, org_code, item_code) for traceability.

- Every dimension has a **primary key**; every fact has **foreign keys** to its dimensions (26 in total), declared as real constraints so ERD tools draw the schema and the database enforces referential integrity.
- Measures use `numeric` (exact money and counts) or `integer`; elapsed times are `numeric(…, 2)` hours; flags are `boolean`.
- Degenerate dimensions stay on the fact (`sitrep_no`, `distribution_id` + `line_no`, `request_id`, `priority`).

Performance. Dashboards read the marts (materialized views), not the facts: about 78× faster for the event summary. At the current volume every analytical query runs in well under 50 ms with sequential scans (see query-performance.md). The scale-up path is B-tree indexes on the most-filtered fact foreign keys and partitioning facts by event year.

Security objects. Row-level security policies on all four facts, grants declared in dbt (re-applied on every build), and the `governance.my_regions` view (see governance.md).

ER diagrams

Generated from the database catalog by `docs/build/diagrams.py`, so they always match the deployed schema.

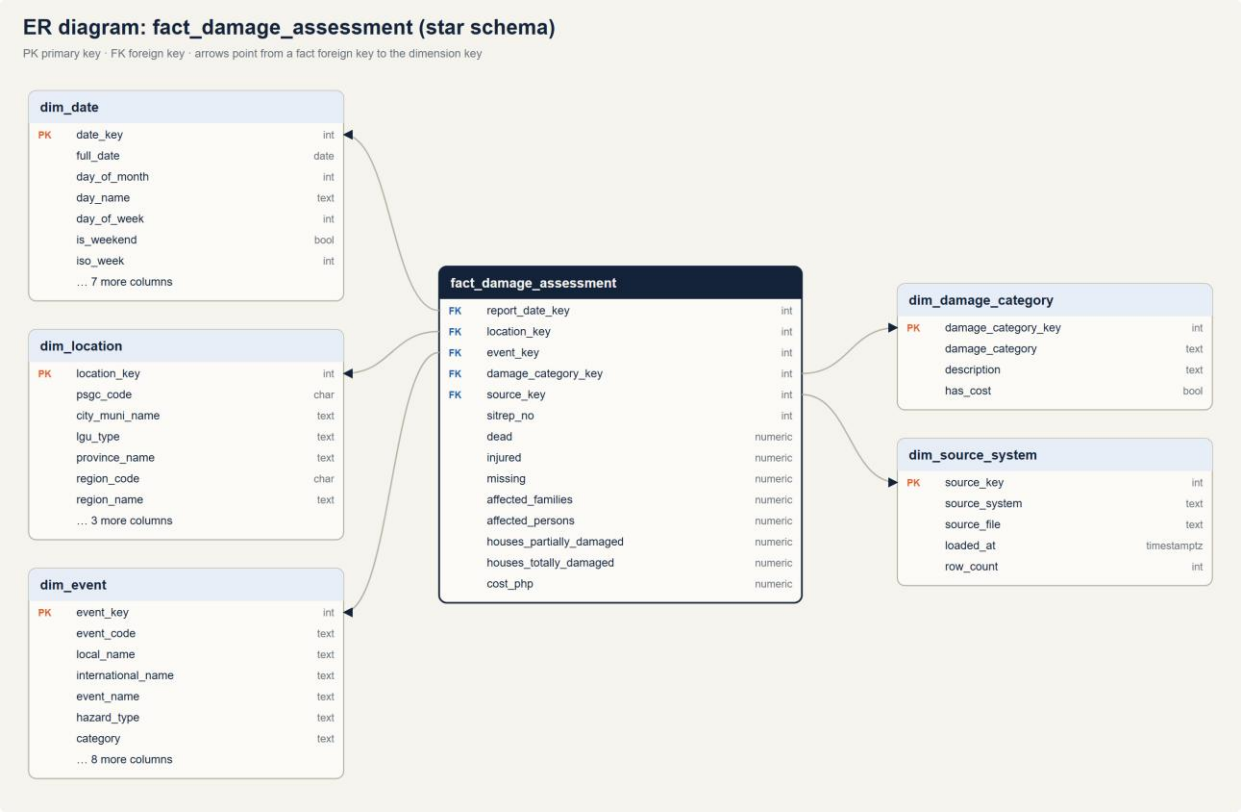


Figure: ER diagram: damage assessment star

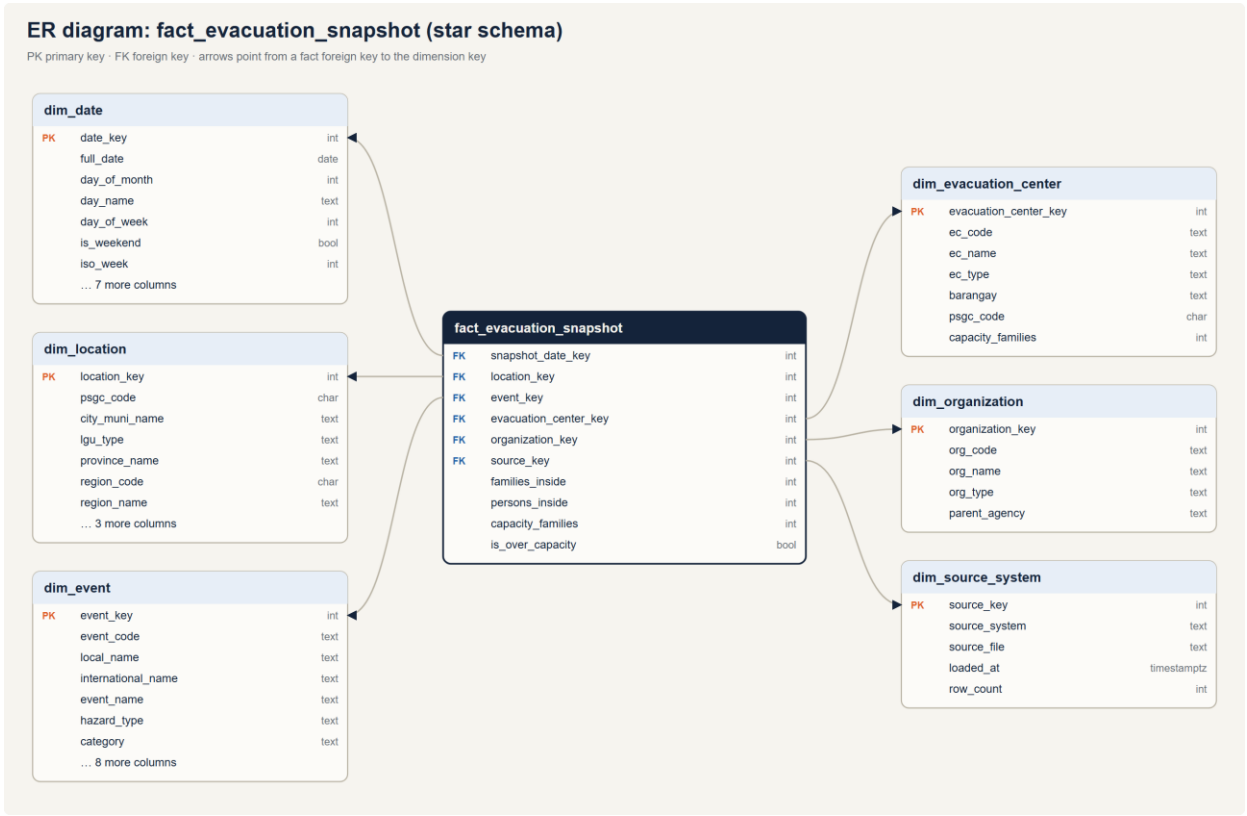


Figure: ER diagram: evacuation snapshot star

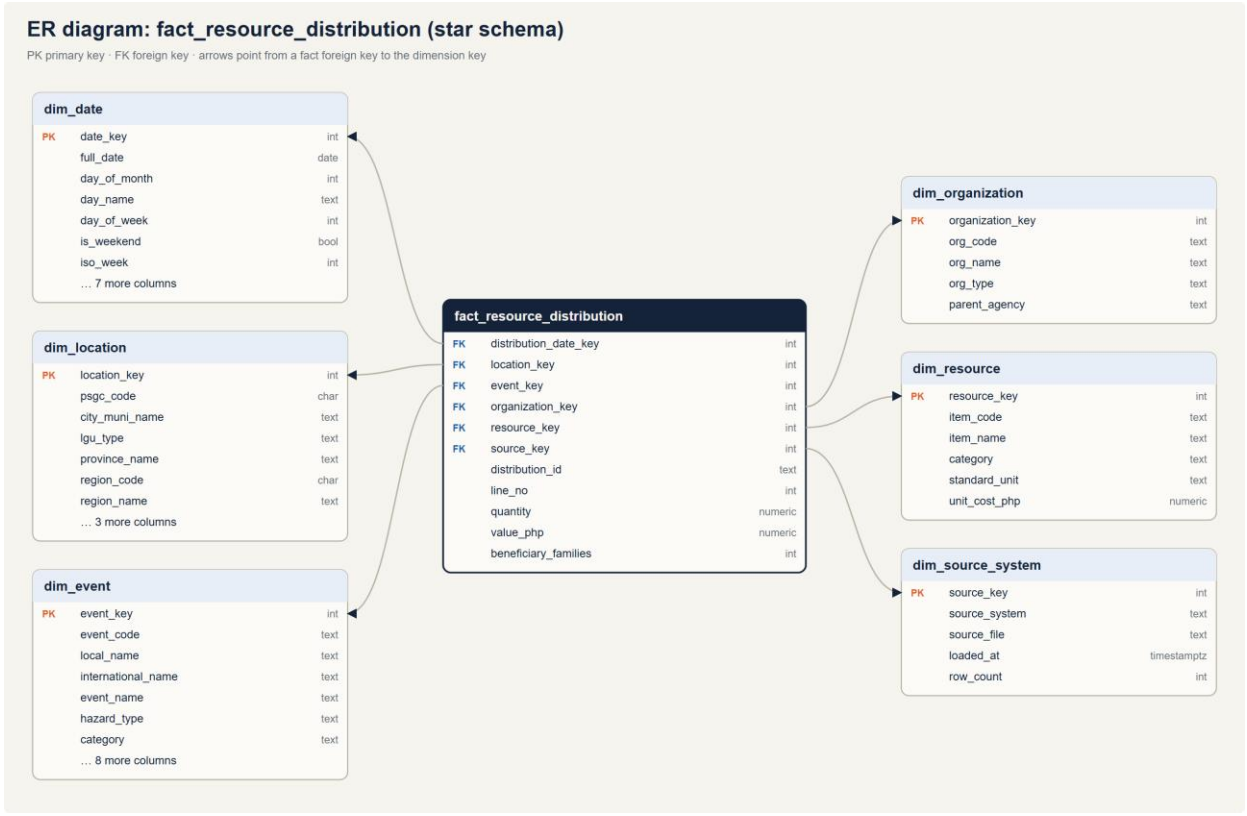


Figure: ER diagram: resource distribution star

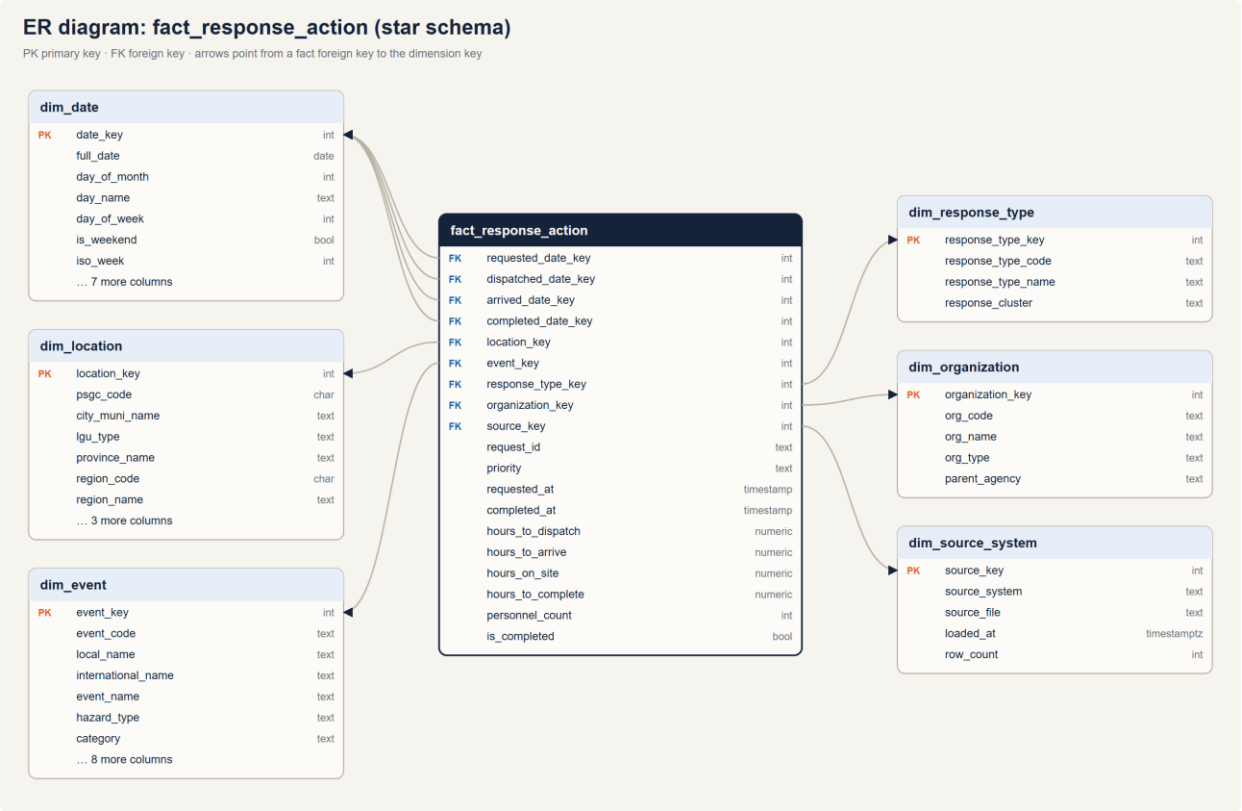


Figure: ER diagram: response action star

2.4 DRIW Dimensional Model

Star schema in the `dw` schema, built by dbt from the cleansed staging layer (only rows with `dq_status` valid or warning). All facts share conformed dimensions, so damage, evacuation, relief and response can be analyzed side by side for the same event and place.

Fact tables

Fact	Type	Grain (one row per…)	Rows *	Measures
`fact_damage_assessment`	Transaction	event × city/municipality × damage category	8,806	dead, injured, missing, affected families/persons, houses partially/totally damaged, cost (PHP)
`fact_evacuation_snapshot`	Periodic snapshot	evacuation center × day × event	31,110	families inside, persons inside, capacity, over-capacity flag
`fact_resource_distribution`	Transaction	distribution line item	11,179	quantity (standard unit), value (PHP), beneficiary families
`fact_response_action`	Accumulating snapshot	response request	32,347	hours to dispatch / arrive / on site / complete, personnel, completed flag

* Seed 271.

Design decisions

- **Damage uses the latest sitrep only.** NDRRMC situation reports are cumulative, so summing every sitrep would double-count. The fact keeps the latest *valid* sitrep per event × place × category.
- **Evacuation grain includes the event.** Overlapping events (Nika, Ofel and Pepito, Nov 2024) can fill the same center on the same day.
- **Evacuation measures are semi-additive.** Sum across centers or places for one day; across days use the peak (MAX of daily totals) or the average, never the sum.

- **Header measures stay additive.** `beneficiary_families` belongs to a whole distribution, so it is stored on the record's first valid line only.
- **Open milestones.** A response request not yet completed has `completed_date_key = -1` ("Not yet occurred"), so foreign keys are never null; a later load fills it in.
- **Degenerate dimensions:** `sitrep_no`, `distribution_id` + `line_no`, `request_id`, `priority`.

Dimensions

Dimension	Natural key	Rows	Notes
`dim_date`	calendar date	2,923	2019–2026 + member −1. Month, quarter, year, weekend, typhoon-season flag. Role-plays four times in `fact_response_action`.
`dim_location`	PSGC code	1,671	Region → province → city/municipality. **SCD Type 2** (see below).
`dim_event`	event code (`TC2024-KRISTINE`)	31	Local + international name, hazard type, category and rank, max wind, max signal, dates, duration.
`dim_organization`	org code	13	Government, NGO, international, LGU; source name variants resolved via aliases.
`dim_resource`	item code	10	Category (Food, Water, Non-Food, Medical, Shelter), standard unit, unit cost.
`dim_evacuation_center`	EC code	1,751	Type, barangay, capacity, PSGC of its city/municipality. SCD Type 1.
`dim_damage_category`	category	4	Population, Housing, Infrastructure, Agriculture.
`dim_response_type`	type code	5	Response type and cluster (life-saving, logistics, access).
`dim_source_system`	batch id	per load	Lineage: source system and file each fact row came from.

SCD Type 2: dim_location

Provinces change. On 17 September 2022 a plebiscite ratified **RA 11550**, splitting Maguindanao into *Maguindanao del Norte* and *Maguindanao del Sur*. The change log `dbt/seeds/psgc_changes.csv` records each affected city/municipality with its new province and effective date (2022-09-18). `dim_location` turns it into versions:

psgc_code	city_muni_name	province_name	valid_from	valid_to	is_current
1903811000	Parang	Maguindanao	1900-01-01	2022-09-17	false
1903811000	Parang	Maguindanao del Norte	2022-09-18	9999-12-31	true

Facts join to the version **valid on the fact's own date**, so Typhoon Paeng (Oct 2022) reports under the new provinces, while any earlier event would still roll up to "Maguindanao". New PSGC releases are handled by appending rows to the change log.

Bus matrix

Fact \ Dimension	Date	Location	Event	Organization	Resource	Evac Center	Damage Cat	Response Type	Source
Damage Assessment	✓	✓	✓				✓		✓
Evacuation Snapshot	✓	✓	✓	✓		✓			✓
Resource Distribution	✓	✓	✓	✓	✓				✓
Response Action	✓ ×4	✓	✓	✓				✓	✓

Physical design and tests

Every dimension has a primary key and every fact has foreign keys to its dimensions (26 in total), so ERD tools such as pgAdmin or DBeaver draw the star directly. dbt tests check that:

- keys are unique and not null, and each fact has one row per declared grain;
- every row that passed data quality reaches its fact, so inner joins drop nothing silently;
- SCD2 versions never overlap or leave gaps, and exactly one version per place is current;
- Paeng's damage in the former Maguindanao rolls up to the new provinces.

Data marts (mart schema)

Materialized views refreshed by every `dbt build`, each shaped for a dashboard. A dbt test checks that every mart adds back up to the facts it summarizes.

Mart	Grain	Used by
`mart_event_summary`	event	Executive: headline figures from all four facts
`mart_impact_cube`	`GROUP BY CUBE (year, region, hazard)`: every subtotal, labelled by `grouping_level`	Executive: region totals
`mart_province_event_impact`	event × province (SCD2 province at the time)	Logistics: relief vs impact
`mart_evacuation_daily`	event × day (sum across centers only)	Operations
`mart_response_performance`	`GROUPING SETS` over event, response type, priority (`All` = rolled up)	Operations
`mart_relief_by_org_category`	event × organization × resource category	Logistics

The dashboards read these marts live through the PHP API on XAMPP (`/driw/api/marts.php`), as the read-only role `driw_web`. A JSON snapshot of the marts (`ingestion/export_marts.py` → `web/src/data/`) is bundled as a fallback for when the warehouse is unreachable.

Example: OLAP with GROUP BY CUBE

```
select d.year_number, r.category, sum(f.value_php) as relief_value_php
from dw.fact_resource_distribution f
join dw.dim_date d on d.date_key = f.distribution_date_key
join dw.dim_resource r using (resource_key)
group by cube (d.year_number, r.category);      -- every subtotal plus the grand total
```

3. Data Dictionary and Metadata

Generated from the live PostgreSQL catalog by `docs/build/data_dictionary.py` on 2026-10-16 12:52. Names, types, keys and nullability come from the database; descriptions are curated. Row counts are from the current build (seed 271).

Last successful load: 2026-10-15 18:43.

3.1 Schemas (layers)

Schema	Purpose	Objects
`raw`	Landing zone. Source extracts exactly as received, every business column as text, plus lineage columns `_batch_id`, `_source_file`, `_row_number`. Loaded by `ingestion/load_raw.py` (full reload).	5
`ref`	Master and reference data: PSGC geography, canonical organizations and relief items, and the alias maps that conform source spellings to them.	7
`meta`	Pipeline metadata: one row per loaded file (`load_batch`), applied migrations, and the defect manifest written by the synthetic generator.	3
`staging`	Cleansed and conformed source data (dbt). Keeps every raw row and adds `dq_issues` (rules that fired) and `dq_status` (valid, warning, rejected, duplicate).	7
`dq`	Data-quality layer (dbt): rule catalog, issue log, scorecard, quarantine and reconciliation.	5
`dw`	Enterprise star schema (dbt): conformed dimensions and fact tables. Only valid/warning rows load.	13
`mart`	Data marts (dbt materialized views) shaped for the dashboards, including ROLAP aggregates.	6
`secure`	Region-scoped views for regional users; they run with the caller's rights (row-level security).	2

3.2 Lineage

Source system	Source file(s)	Raw table	Staging model	Warehouse fact
NDRRMC situation reports	`ndrrmc/sitreps_<event>.csv`	`raw.ndrrmc_damage_report`	`staging.stg_ndrrmc_damage`	`dw.fact_damage_assessment`
DSWD DROMIC	`dromic/dromic_<event>.xlsx`	`raw.dromic_evacuation`	`staging.stg_dromic_evacuation`	`dw.fact_evacuation_snapshot`
NGO / agency partners	`ngo/distributions_<year>.json`	`raw.ngo_distribution`	`staging.stg_ngo_distribution_line`	`dw.fact_resource_distribution`
PAGASA	`pagasa/tc_bulletins.csv`	`raw.pagasa_event`	`staging.stg_pagasa_event`	`dw.dim_event`
LGU / MDRRMO	`lgu/response_logs.csv`	`raw.lgu_response_log`	`staging.stg_lgu_response`	`dw.fact_response_action`
PSA PSGC (community API)	`psgc_api/*.json`	`ref.psgc`	`staging.stg_location_resolution`	`dw.dim_location`

Every fact row carries `source_key` → `dw.dim_source_system` (the file and batch it came from), and every raw row keeps its `_batch_id`, `_source_file` and `_row_number`.

3.3 dw · Star schema

dw.dim_damage_category

Population, Housing, Infrastructure, Agriculture. *table, 4 rows.*

Column	Type	Key	Null	Description
damage_category_key	integer	PK	no	Surrogate key of dim_damage_category.
damage_category	text		yes	Population, Housing, Infrastructure or Agriculture.
description	text		yes	Description of the category.
has_cost	boolean		yes	Whether the category carries a PHP cost.

dw.dim_date

Calendar, 2019–2026, plus member -1 (“Not yet occurred”) for open milestones. Type 0. *table, 2,923 rows.*

Column	Type	Key	Null	Description
date_key	integer	PK	no	Surrogate key of dim_date, YYYYMMDD (-1 = not yet occurred).
full_date	date		yes	Calendar date.
day_of_month	integer		yes	Day of the month, 1–31.
day_name	text		yes	Weekday name.
day_of_week	integer		yes	ISO weekday, 1 (Monday) to 7 (Sunday).
is_weekend	boolean		yes	Saturday or Sunday.
iso_week	integer		yes	ISO week number.
month_number	integer		yes	Month, 1–12.
month_name	text		yes	Month name.
year_month	text		yes	YYYY-MM.
quarter_number	integer		yes	Quarter, 1–4.
quarter_name	text		yes	Q1–Q4.
year_number	integer		yes	Calendar year.
is_typhoon_season	boolean		yes	June–November.

dw.dim_evacuation_center

Evacuation centers from DROMIC reports. SCD Type 1 (latest attributes). *table, 1,751 rows.*

Column	Type	Key	Null	Description
evacuation_center_key	integer	PK	no	Surrogate key of dim_evacuation_center.
ec_code	text		yes	Evacuation-center code from DROMIC (natural key).
ec_name	text		yes	Evacuation-center name.
ec_type	text		yes	School, covered court, gymnasium, barangay hall or church.
barangay	text		yes	Barangay where the center is located.
psgc_code	character		yes	Philippine Standard Geographic Code, 10 digits (natural key).
capacity_families	integer		yes	Families the evacuation center can hold.

dw.dim_event

Hazard events (tropical cyclones and monsoon floods) from PAGASA bulletins. *table, 31 rows.*

Column	Type	Key	Null	Description
event_key	integer	PK	no	Surrogate key of dim_event.
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
local_name	text		yes	PAGASA local storm name.

international_name	text		yes	International (JMA) storm name.
event_name	text		yes	Local name with international name, e.g. Kristine (Trami).
hazard_type	text		yes	Tropical Cyclone or Flood (southwest monsoon).
category	text		yes	Category: storm category for events, resource category for relief items.
category_rank	integer		yes	Storm category as a number, 1 (tropical depression) to 5 (super typhoon).
max_wind_kph	integer		yes	Maximum sustained winds, km/h.
max_signal	integer		yes	Highest tropical cyclone wind signal raised.
start_date	date		yes	First day of the event.
end_date	date		yes	Last day of the event.
duration_days	integer		yes	Event length in days, inclusive.
event_year	integer		yes	Year the event started.
affected_areas	text		yes	Provinces named in the PAGASA bulletin, as published.

dw.dim_location

City/municipality, keyed on PSGC code. SCD Type 2: one row per period of unchanged attributes (valid_from–valid_to). *table, 1,671 rows.*

Column	Type	Key	Null	Description
location_key	integer	PK	no	Surrogate key of dim_location (the SCD2 version valid on the fact's date).
psgc_code	character		yes	Philippine Standard Geographic Code, 10 digits (natural key).
city_muni_name	text		yes	Official PSGC name of the city or municipality.
lgu_type	text		yes	City or Municipality.
province_name	text		yes	Province in force on the fact's date (e.g. Maguindanao del Sur after Sep 2022).
region_code	character(10)		yes	PSGC region code.
region_name	text		yes	Region name.
valid_from	date		yes	SCD2: first day this version applies.
valid_to	date		yes	SCD2: last day this version applies (9999-12-31 = current).
is_current	boolean		yes	Whether this SCD2 version is the current one.

dw.dim_organization

Responding and reporting organizations (government, NGO, international, LGU). *table, 13 rows.*

Column	Type	Key	Null	Description
organization_key	integer	PK	no	Surrogate key of dim_organization.
org_code	text		yes	Organization code (natural key).
org_name	text		yes	Organization name.
org_type	text		yes	National Government, NGO, International or LGU.
parent_agency	text		yes	Department the organization belongs to.

dw.dim_resource

Relief items with category, standard unit and unit cost. *table, 10 rows.*

Column	Type	Key	Null	Description
resource_key	integer	PK	no	Surrogate key of dim_resource.
item_code	text		yes	Canonical relief item code.
item_name	text		yes	Relief item name.
category	text		yes	Category: storm category for events, resource category for relief items.
standard_unit	text		yes	Unit all quantities are converted to (pack, kg, liter, kit,

				piece).
unit_cost_php	numeric(12,2)		yes	Indicative unit cost, PHP.

dw.dim_response_type

Response request types and their cluster. *table, 5 rows.*

Column	Type	Key	Null	Description
response_type_key	integer	PK	no	Surrogate key of dim_response_type.
response_type_code	text		yes	Response type code, e.g. RESCUE.
response_type_name	text		yes	Response type name.
response_cluster	text		yes	Life-saving, Logistics, or Access and infrastructure.

dw.dim_source_system

Lineage: one row per loaded source file (batch). *table, 420 rows.*

Column	Type	Key	Null	Description
source_key	integer	PK	no	Surrogate key of dim_source_system (lineage to the loaded file).
source_system	text		yes	NDRRMC, DSWD_DROMIC, NGO, PAGASA or LGU.
source_file	text		yes	Source file name.
loaded_at	timestamp with time zone		yes	When the batch was loaded.
row_count	integer		yes	Rows in the source file.

dw.fact_damage_assessment

Grain: event × city/municipality × damage category (latest valid sitrep). Transaction fact. *table, 8,806 rows.*

Column	Type	Key	Null	Description
report_date_key	integer	FK → dim_date	yes	Date of the sitrep the figures come from (dim_date).
location_key	integer	FK → dim_location	yes	Surrogate key of dim_location (the SCD2 version valid on the fact's date).
event_key	integer	FK → dim_event	yes	Surrogate key of dim_event.
damage_category_key	integer	FK → dim_damage_category	yes	Surrogate key of dim_damage_category.
source_key	integer	FK → dim_source_system	yes	Surrogate key of dim_source_system (lineage to the loaded file).
sitrep_no	integer		yes	NDRRMC situation-report number (degenerate dimension).
dead	numeric		yes	Deaths.
injured	numeric		yes	Injured persons.
missing	numeric		yes	Missing persons.
affected_families	numeric		yes	Families affected (NDRRMC).
affected_persons	numeric		yes	Persons affected (NDRRMC).
houses_partially_damaged	numeric		yes	Houses partially damaged.
houses_totally_damaged	numeric		yes	Houses totally destroyed.
cost_php	numeric		yes	Damage cost, PHP (Housing, Infrastructure, Agriculture rows).

dw.fact_evacuation_snapshot

Grain: evacuation center × day × event. Periodic snapshot; occupancy measures are semi-additive (never sum across days). *table, 31,110 rows.*

Column	Type	Key	Null	Description

snapshot_date_key	integer	FK → dim_date	yes	Date of the snapshot (dim_date).
location_key	integer	FK → dim_location	yes	Surrogate key of dim_location (the SCD2 version valid on the fact's date).
event_key	integer	FK → dim_event	yes	Surrogate key of dim_event.
evacuation_center_key	integer	FK → dim_evacuation_center	yes	Surrogate key of dim_evacuation_center.
organization_key	integer	FK → dim_organization	yes	Surrogate key of dim_organization.
source_key	integer	FK → dim_source_system	yes	Surrogate key of dim_source_system (lineage to the loaded file).
families_inside	integer		yes	Families inside the center (semi-additive).
persons_inside	integer		yes	Persons inside the center (semi-additive).
capacity_families	integer		yes	Families the evacuation center can hold.
is_over_capacity	boolean		yes	Families inside exceeded the center's capacity.

dw.fact_resource_distribution

Grain: one relief item in one distribution. Transaction fact; quantities in standard units. *table, 11,179 rows.*

Column	Type	Key	Null	Description
distribution_date_key	integer	FK → dim_date	yes	Date of the distribution (dim_date).
location_key	integer	FK → dim_location	yes	Surrogate key of dim_location (the SCD2 version valid on the fact's date).
event_key	integer	FK → dim_event	yes	Surrogate key of dim_event.
organization_key	integer	FK → dim_organization	yes	Surrogate key of dim_organization.
resource_key	integer	FK → dim_resource	yes	Surrogate key of dim_resource.
source_key	integer	FK → dim_source_system	yes	Surrogate key of dim_source_system (lineage to the loaded file).
distribution_id	text		yes	Source distribution identifier (degenerate dimension).
line_no	integer		yes	Line number within the distribution.
quantity	numeric		yes	Quantity in the item's standard unit.
value_php	numeric		yes	Value of the line, PHP.
beneficiary_families	integer		yes	Families served by the whole distribution; stored on its first line only so it stays additive.

dw.fact_response_action

Grain: one response request. Accumulating snapshot with four milestone dates. *table, 32,347 rows.*

Column	Type	Key	Null	Description
requested_date_key	integer	FK → dim_date	yes	Date of the request (role-playing dim_date).
dispatched_date_key	integer	FK → dim_date	yes	Date a unit was dispatched (role-playing dim_date).
arrived_date_key	integer	FK → dim_date	yes	Date the responding unit arrived (role-playing dim_date; -1 = not yet).
completed_date_key	integer	FK → dim_date	yes	Date the request was completed (role-playing dim_date; -1 = not yet).
location_key	integer	FK → dim_location	yes	Surrogate key of dim_location (the SCD2 version valid on the fact's date).
event_key	integer	FK → dim_event	yes	Surrogate key of dim_event.
response_type_key	integer	FK → dim_response_type	yes	Surrogate key of dim_response_type.
organization_key	integer	FK → dim_organization	yes	Surrogate key of dim_organization.
source_key	integer	FK → dim_source_system	yes	Surrogate key of dim_source_system (lineage to the loaded file).
request_id	text		yes	Source request identifier (degenerate dimension).
priority	text		yes	P1 critical, P2 urgent, P3 routine ('All' in roll-up rows).
requested_at	timestamp without time zone		yes	Timestamp of the request.
completed_at	timestamp without time zone		yes	Timestamp the request was completed (null while open).

hours_to_dispatch	numeric		yes	Hours from request to dispatch.
hours_to_arrive	numeric		yes	Hours from dispatch to arrival.
hours_on_site	numeric		yes	Hours from arrival to completion.
hours_to_complete	numeric		yes	Hours from request to completion.
personnel_count	integer		yes	Personnel sent on the request.
is_completed	boolean		yes	Whether the request has been completed.

3.4 mart · Data marts

mart.mart_evacuation_daily

Evacuation-center load per event per day (summed across centers only). *materialized view, 372 rows.*

Column	Type	Key	Null	Description
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
snapshot_date	date		yes	Date of the daily snapshot.
day_of_event	integer		yes	Days since the event started (negative = pre-emptive evacuation).
centers_open	bigint		yes	Evacuation centers with evacuees that day.
families_inside	bigint		yes	Families inside the center (semi-additive).
persons_inside	bigint		yes	Persons inside the center (semi-additive).
capacity_families	bigint		yes	Families the evacuation center can hold.
centers_over_capacity	bigint		yes	Centers where families inside exceeded capacity that day.
occupancy_rate	numeric		yes	Families inside divided by capacity, across open centers.

mart.mart_event_summary

One row per event: headline damage, evacuation, relief and response figures. *materialized view, 31 rows.*

Column	Type	Key	Null	Description
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
event_name	text		yes	Local name with international name, e.g. Kristine (Trami).
local_name	text		yes	PAGASA local storm name.
international_name	text		yes	International (JMA) storm name.
hazard_type	text		yes	Tropical Cyclone or Flood (southwest monsoon).
category	text		yes	Category: storm category for events, resource category for relief items.
category_rank	integer		yes	Storm category as a number, 1 (tropical depression) to 5 (super typhoon).
max_signal	integer		yes	Highest tropical cyclone wind signal raised.
start_date	date		yes	First day of the event.
end_date	date		yes	Last day of the event.
event_year	integer		yes	Year the event started.
dead	numeric		yes	Deaths.
injured	numeric		yes	Injured persons.
missing	numeric		yes	Missing persons.
affected_families	numeric		yes	Families affected (NDRRMC).
affected_persons	numeric		yes	Persons affected (NDRRMC).
houses_partially_damaged	numeric		yes	Houses partially damaged.
houses_totally_damaged	numeric		yes	Houses totally destroyed.
damage_cost_php	numeric		yes	Total damage cost, PHP.
housing_cost_php	numeric		yes	Housing damage, PHP.

infrastructure_cost_php	numeric		yes	Infrastructure damage, PHP.
agriculture_cost_php	numeric		yes	Agricultural damage, PHP.
cities_municipalities_affected	bigint		yes	Distinct cities/municipalities with reported damage.
provinces_affected	bigint		yes	Distinct provinces with reported damage.
peak_persons_in_centers	bigint		yes	Highest daily total of persons in centers (sum across centers, max across days).
peak_families_in_centers	bigint		yes	Highest daily total of families in centers.
days_with_evacuees	bigint		yes	Days on which any center held evacuees.
centers_used	bigint		yes	Distinct evacuation centers used during the event.
over_capacity_center_days	bigint		yes	Center-days in which a center was over capacity.
relief_value_php	numeric		yes	Relief value, PHP (standard quantity × unit cost).
families_served	bigint		yes	Sum of beneficiary families over distributions (a family can be served more than once).
distributions	bigint		yes	Number of distinct distributions.
relief_php_per_affected_person	numeric		yes	Relief value divided by persons affected, PHP.
response_requests	bigint		yes	Number of response requests.
median_hours_to_dispatch	numeric		yes	Median hours from request to dispatch.
median_hours_to_arrive	numeric		yes	Median hours from dispatch to arrival.
completion_rate	numeric		yes	Share of requests completed, 0–1.

mart.mart_impact_cube

GROUP BY CUBE over year × region × hazard type: every subtotal, labelled by grouping_level. *materialized view, 160 rows.*

Column	Type	Key	Null	Description
grouping_level	text		yes	Which dimensions a CUBE row is broken down by: year, region, hazard, combinations, or grand_total.
event_year	integer		yes	Year the event started.
region_name	text		yes	Region name.
hazard_type	text		yes	Tropical Cyclone or Flood (southwest monsoon).
events	bigint		yes	Number of distinct events.
dead	numeric		yes	Deaths.
affected_persons	numeric		yes	Persons affected (NDRRMC).
houses_totally_damaged	numeric		yes	Houses totally destroyed.
damage_cost_php	numeric		yes	Total damage cost, PHP.

mart.mart_province_event_impact

Event × province impact next to relief delivered (coverage). *materialized view, 168 rows.*

Column	Type	Key	Null	Description
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
event_name	text		yes	Local name with international name, e.g. Kristine (Trami).
event_year	integer		yes	Year the event started.
region_name	text		yes	Region name.
province_name	text		yes	Province in force on the fact's date (e.g. Maguindanao del Sur after Sep 2022).
dead	numeric		yes	Deaths.
affected_persons	numeric		yes	Persons affected (NDRRMC).
affected_families	numeric		yes	Families affected (NDRRMC).
houses_totally_damaged	numeric		yes	Houses totally destroyed.
damage_cost_php	numeric		yes	Total damage cost, PHP.

relief_value_php	numeric		yes	Relief value, PHP (standard quantity × unit cost).
families_served	bigint		yes	Sum of beneficiary families over distributions (a family can be served more than once).
relief_php_per_affected_person	numeric		yes	Relief value divided by persons affected, PHP.
family_coverage_ratio	numeric		yes	Family servings divided by affected families.

mart.mart_relief_by_org_category

Relief delivered per event × organization × resource category. *materialized view, 730 rows.*

Column	Type	Key	Null	Description
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
org_code	text		yes	Organization code (natural key).
org_name	text		yes	Organization name.
org_type	text		yes	National Government, NGO, International or LGU.
resource_category	text		yes	Food, Water, Non-Food, Medical or Shelter.
distributions	bigint		yes	Number of distinct distributions.
line_items	bigint		yes	Number of distribution lines.
relief_value_php	numeric		yes	Relief value, PHP (standard quantity × unit cost).
families_served	bigint		yes	Sum of beneficiary families over distributions (a family can be served more than once).

mart.mart_response_performance

Response timeliness with GROUPING SETS roll-ups; 'All' marks a rolled-up value. *materialized view, 671 rows.*

Column	Type	Key	Null	Description
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
response_type	text		yes	Response type name ('All' in roll-up rows).
priority	text		yes	P1 critical, P2 urgent, P3 routine ('All' in roll-up rows).
requests	bigint		yes	Number of requests.
median_hours_to_dispatch	numeric		yes	Median hours from request to dispatch.
p90_hours_to_dispatch	numeric		yes	90th-percentile hours from request to dispatch.
median_hours_to_arrive	numeric		yes	Median hours from dispatch to arrival.
median_hours_to_complete	numeric		yes	Median hours from request to completion.
completion_rate	numeric		yes	Share of requests completed, 0–1.
personnel_deployed	bigint		yes	Total personnel sent.

3.5 Supporting layers

Column-level detail for these layers is in the database itself; summarized here.

Object	Kind	Rows	Columns
raw.dromic_evacuation	table	31,241	report_date, disaster_name, region, province, municipality, barangay, ec_code, ec_name, ec_type, capacity_families, families_inside, persons_inside, _batch_id, _source_file, _row_number
raw.lgu_response_log	table	32,749	request_id, incident, province, lgu_name, request_type, priority, requested_at, dispatched_at, arrived_at, completed_at, responding_unit, personnel_count, _batch_id, _source_file, _row_number
raw.ndrrmc_damage_report	table	16,151	sitrep_no, report_date, incident_name, region, province, city_muni,

			damage_category, dead, injured, missing, affected_families, affected_persons, houses_partial, houses_total, cost_php, remarks, _batch_id, _source_file, _row_number
raw.ngo_distribution	table	5,661	record, _batch_id, _source_file, _row_number
raw.pagasa_event	table	39	local_name, international_name, category, max_wind_kph, max_signal, start_date, end_date, affected_areas, _batch_id, _source_file, _row_number
ref.location_alias	table	0	alias_name, alias_scope, psgc_code, match_method
ref.organization	table	13	org_code, org_name, org_type, parent_agency
ref.organization_alias	table	42	alias_name, org_code
ref.psgc	table	43,793	psgc_code, name, geographic_level, region_code, province_code, city_muni_code, city_class, income_class, urban_rural, population, population_census, publication, province_name
ref.psgc_changes	table	37	psgc_code, city_muni_name, attribute, old_value, new_value, effective_date, reference
ref.resource_item	table	10	item_code, item_name, category, standard_unit, unit_cost_php
ref.resource_item_alias	table	25	source_item_code, item_code, source_unit, to_standard_factor
meta.defect_manifest	table	19	source_system, defect, planted_count, seed, loaded_at
meta.load_batch	table	420	batch_id, source_system, source_file, row_count, started_at, finished_at, status
meta.schema_migration	table	5	version, applied_at
staging.stg_dromic_evacuation	table	31,241	_batch_id, _source_file, _row_number, raw_report_date, date_format, dominant_date_format, report_date, disaster_name, province, raw_municipality, barangay, raw_ec_code, ec_name, ec_type, capacity_families, families_inside, persons_inside, psgc_code, location_match_method, event_key, ec_code, dq_issues, dq_status
staging.stg_event_alias	table	124	event_key, alias, start_date, end_date
staging.stg_lgu_response	table	32,749	_batch_id, _source_file, _row_number, request_id, incident, province, raw_lgu_name, raw_request_type, request_type, priority, requested_at, dispatched_at, arrived_at, completed_at, responding_unit, personnel_count, psgc_code, location_match_method, event_key, hours_to_dispatch, hours_to_arrive, hours_on_site, hours_to_complete, dq_issues, dq_status
staging.stg_location_resolution	table	2,969	raw_province, raw_name, psgc_code, official_name, match_method, edit_distance
staging.stg_ndrrmc_damage	table	16,151	_batch_id, _source_file, _row_number, copy_number, sitrep_no, report_date, incident_name, province, raw_city_muni, damage_category, dead, injured, missing, affected_families, affected_persons, houses_partial, houses_total, cost_php, remarks, raw_dead, raw_injured,

			raw_missing, raw_affected_families, raw_affected_persons, raw_houses_partial, raw_houses_total, raw_cost_php, psgc_code, location_match_method, event_key, dq_issues, dq_status
staging.stg_ngo_distribution_line	table	11,529	_batch_id, _source_file, _row_number, line_no, distribution_id, distributed_on, event_key, raw_event_ref, org_code, raw_organization, psgc_code, province, raw_municipality, location_match_method, beneficiary_families, item_code, source_item_code, source_unit, source_quantity, to_standard_factor, standard_unit, quantity_standard, value_php, header_issues, line_issues, dq_issues, dq_status
staging.stg_pagasa_event	table	39	_batch_id, _source_file, _row_number, local_name, international_name, category, max_wind_kph, max_signal, start_date, end_date, affected_areas, event_key, copy_number, dq_issues, dq_status
dq.dq_issue_log	table	17,722	source_system, rule_code, field_name, severity, _batch_id, _source_file, _row_number, line_no
dq.dq_quarantine	table	925	source_system, _source_file, _row_number, line_no, reject_reasons, all_issues, batch_source, batch_loaded_at
dq.dq_reconciliation	table	19	source_system, planted_defect, detected_by, planted_count, detected_count, reconciliation_status
dq.dq_rule_summary	table	36	source_system, rule_code, severity, action, description, issue_count, affected_rows, source_rows, pct_rows_affected
dq.dq_rules	table	36	source_system, rule_code, severity, action, description, planted_defect
secure.regional_event_impact	view	168	event_code, event_name, region_name, province_name, dead, affected_persons, houses_totally_damaged, damage_cost_php, relief_value_php, family_servings
secure.regional_response_summary	view	450	event_code, event_name, region_name, response_type, requests, median_hours_to_dispatch, completion_rate

4. ETL Design

4.1 Overview

The ETL is a set of **custom Python and SQL scripts**: Python for extraction and loading, and **dbt** for transformation, data-quality rules and tests. One command runs the whole pipeline:

```
.venv/Scripts/python ingestion/pipeline.py      # migrate → reference → psgc → generate → raw → dbt → export
```

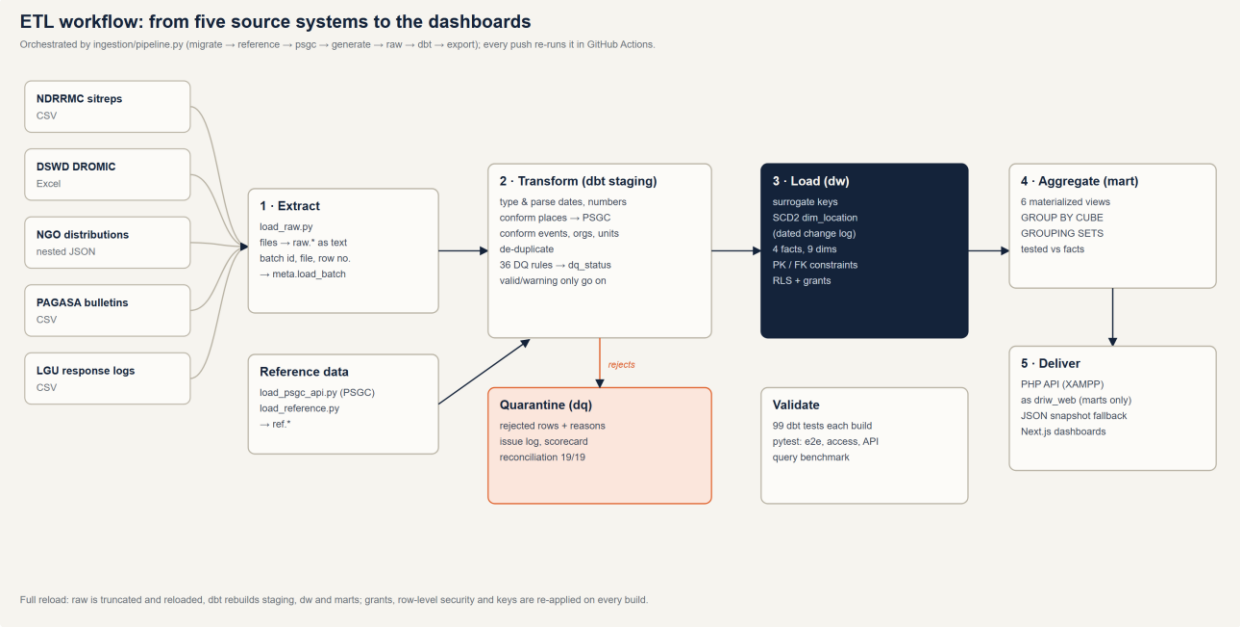


Figure: ETL workflow

4.2 Data sources

Source system	Format	Files	Rows landed	Typical defects (planted and counted)
NDRRMC situation reports	CSV	31 (one per event)	16,151	duplicate rows, blanks and "N/A", thousands separators, negative costs, swapped figures, misspelled towns
DSWD DROMIC evacuation reports	Excel (.xlsx)	31	31,241	mixed date formats per field office, missing center codes, overcrowding, misspelled towns
NGO / agency relief distributions	nested JSON	6 (one per year)	5,661 records	organization aliases, partner item codes and units, missing locations, unknown items
PAGASA bulletins	CSV	1	39	re-issued (duplicate) bulletins, local vs international names
LGU / MDRRMO response logs	CSV	1	32,749	non-standard request types, arrival before dispatch, open requests, misspelled towns
PSA PSGC geography	JSON (community API mirror of PSA codes)	6	43,793 units	— (master data)
Hand-curated reference data	CSV	3	events, 13 organizations, 10 relief items	—

Mock vs real. Event names, dates and geography are real; operational figures are synthetic, produced by `ingestion/generate_sources.py` with seed 271 (deterministic). The generator records each defect it plants in `_defect_manifest.json` so the transformation can be proven against it.

4.3 Extract (raw layer)

- `load_raw.py` reads each file in its native format (csv module, openpyxl, json) and bulk-loads it with PostgreSQL `COPY` into `raw.<source>`.
- **Every business column is stored as text**, so values like `"1,234"`, `"N/A"` or `"23-Oct-2024"` are never rejected at load: nothing is lost before the quality rules run.
- Each file becomes a batch in `meta.load_batch`; each row keeps `_batch_id`, `_source_file` and `_row_number` (lineage).
- Strategy: **full reload** (truncate and reload), appropriate for a batch analytical warehouse of this size and simplest to make repeatable.

4.4 Transform (staging and data-quality layers, dbt)

Cleaning

- Parse numbers (strip thousands separators; `"N/A"`, `"-"`, `"TBD"` → null), dates in four formats (ISO, US, `DD-Mon-YYYY`, Excel serial numbers) and timestamps.
- Trim and normalize text; map non-standard codes (`RESQ` → `RESCUE`, `Medical` → `MEDEVAC`).
- Remove exact duplicates (sitrep rows, re-issued bulletins).
- Impute a missing evacuation-center code when the same place, name and barangay always carry one code.

Integration (conformance)

- **Places → PSGC** (`stg_location_resolution`): exact name, then normalized name, then fuzzy match (Levenshtein ≤ 2) within the reported province; ties are left for a steward. 0 false matches against ground truth across 63,347 rows.
- **Events**: local name, international name, `TY <NAME>` or code, matched within the event's time window (PAGASA reuses names).
- **Organizations and relief items**: alias tables in `ref`; partner quantities converted to a standard unit (e.g. 1 sack of rice = 50 kg).
- NGO JSON is flattened from one record per distribution to one row per item line.

Data-quality rules. 36 rules in `dq_rules` with severity *info* (corrected), *warning* (kept, flagged) or *reject* (quarantined). Every staging row gets `dq_issues` and `dq_status`; staging never drops a row. Details: data-quality-framework.md.

Aggregation. Facts keep detail at their grain; aggregation happens in the marts (materialized views) with `GROUP BY CUBE` and `GROUPING SETS`, and in dashboard queries.

4.5 Load (warehouse and marts)

- **Dimensions** load first: surrogate keys by `row_number()`; `dim_location` built as **SCD Type 2** from `ref.psgc` plus a dated change log (`psgc_changes`).
- **Facts** load only *valid* and *warning* rows and look up each dimension key; locations are looked up **as of the fact's date** (SCD2). Damage keeps the latest valid sitrep per event × place × category (sitreps are cumulative).
- Open milestones (a request not yet completed) point to date key -1, so foreign keys are never null.
- After each model: primary/foreign keys, grants and row-level-security policies are re-applied (post-hooks).
- **Marts** refresh as materialized views; `export_marts.py` writes the dashboards' fallback snapshot.

4.6 Orchestration, scheduling and error handling

Concern	Design
Orchestration	`ingestion/pipeline.py` runs the steps in order; any step can be run alone (`pipeline.py dbt export`).
Scheduling	Batch, on demand or by a scheduler (Windows Task Scheduler / cron) running `pipeline.py`; GitHub Actions runs the full pipeline on every push.
Idempotency	Full reload plus dbt's rebuild make every run produce the same warehouse from the same inputs (tested).
Row-level errors	Never stop the load: bad rows go to `dq.dq_quarantine` with their reasons and source file and row.
Step failures	`pipeline.py` stops at the first failing step; `dbt build` fails if any of its 99 tests fail, so a bad build is never published.
Monitoring	`dq.dq_rule_summary` (scorecard), `dq.dq_reconciliation`, `meta.load_batch` (row counts and times per file).

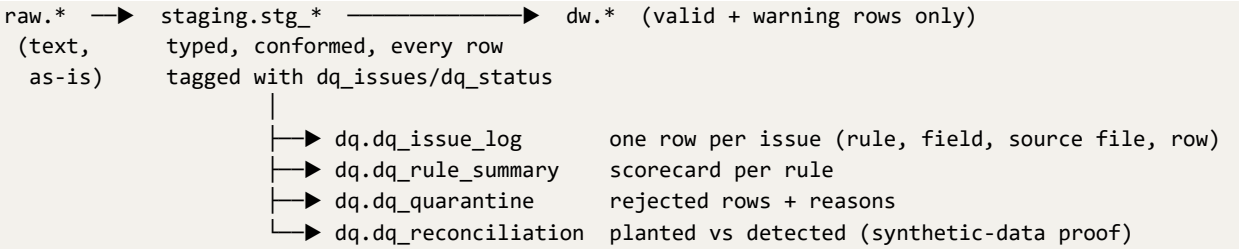
4.7 Tools

Purpose	Tool
Extraction and loading	Python 3.13, psycopg 3 (`COPY`), openpyxl
Transformation, data quality, tests	dbt Core 1.12 with dbt-postgres
Warehouse	PostgreSQL 17
Orchestration and CI	`pipeline.py`, GitHub Actions
Delivery	PHP 8.1 API on XAMPP (Apache), Next.js dashboards

5. Data Quality Framework

How DRIW detects, corrects, flags and quarantines bad source data before it reaches the star schema, and how we prove the rules work.

5.1 Where data quality happens



Staging models never drop rows. Each raw row keeps its lineage (`_batch_id`, `_source_file`, `_row_number`) and gets:

Column	Meaning
`dq_issues`	Array of rule codes that fired, e.g. `{LOC_NAME_VARIANT, MISSING_VALUE:dead}`
`dq_status`	`valid` · `warning` · `rejected` · `duplicate`

5.2 Severity policy

Severity	Action	Reaches the warehouse?	Example
info	Corrected automatically	Yes	`Naga City` → PSGC *City of Naga*; `1,234` → 1234; `R50` sacks → kg
warning	Kept and flagged	Yes	Evacuation center over capacity; required measure blank
reject	Quarantined in `dq.dq_quarantine`	No	Negative cost; arrival before dispatch; unknown item code

The full catalog (36 rules across 5 sources) is `dbt/seeds/dq_rules.csv`. A dbt test fails the build if a staging model emits a rule that is not in the catalog, or if a row's `dq_status` disagrees with the severities of its issues.

5.3 Key techniques

Location conformance (PSGC). Every distinct (province, city/municipality) pair from all sources is resolved once in `stg_location_resolution`, always within the reported province:

1. **Exact** official name.
2. **Normalized**: case, spacing, `ñ`→`n`, `City of X` / `X City`, `Sto.`/`Sta.`.
3. **Fuzzy**: Levenshtein distance ≤ 2 on normalized and letters-only keys. The best candidate must be unique; ties (e.g. `Malniao` is equally close to *Malinao* and *Manito*) stay unmatched for a data steward instead of being guessed.

Accuracy is verified against ground truth embedded in LGU request IDs and DROMIC center codes: **0 false matches** across ~63,000 checked rows (`assert_location_matching_is_accurate`).

Event conformance. Sources refer to events by local name (*Kristine*), international name (*Trami*), `TY KRISTINE` or the key `TC2024-KRISTINE`. `stg_event_alias` lists every form; matches must also fall within the event's time window, because PAGASA reuses names.

Unit standardization. Partner item codes map to canonical items with a conversion factor (`R50`: 1 sack = 50 kg), so quantities and PHP values are comparable across organizations.

Imputation. A missing evacuation-center code is recovered when the same place + center name + barangay always carries a single code elsewhere; otherwise it's rejected.

De-duplication. Exact duplicate sitrep rows and re-issued PAGASA bulletins are marked `duplicate` and excluded.

5.4 Proof: reconciliation against planted defects

The synthetic generator records every defect it plants in `_defect_manifest.json` (loaded to `meta.defect_manifest`). `dq.dq_reconciliation` compares that with detected issues:

Result (seed 271)	
Defect types planted	19
Exactly reconciled	**19 / 19**
Rejected rows not explained by a planted defect	0

Load rates after data quality: NDRRMC 98.2%, DROMIC 99.6%, NGO lines 97.0%, LGU 98.8%.

A warn-level test (`warn_unplanted_rejections`) flags reject rules firing with no planted cause. During development it caught a real parsing bug: monsoon incidents named *"Southwest Monsoon (Habagat) Carina (Gaemi)"* were matched on the first parentheses instead of the last, wrongly rejecting 1,244 rows.

5.5 Running

```
.venv/Scripts/python ingestion/pipeline.py          # everything
.venv/Scripts/python ingestion/pipeline.py dbt      # rebuild staging + dq and run all tests
```

Useful queries:

```
select * from dq.dq_reconciliation;                       -- planted vs detected
select * from dq.dq_rule_summary where issue_count > 0;   -- scorecard
select * from dq.dq_quarantine limit 50;                  -- what was rejected and why
```

6. Data Governance, Security and Privacy

Who may see what in DRIW, how it is enforced inside PostgreSQL, and how it is tested. Everything is enforced by the database itself, so it holds for any client: dashboards, SQL tools, BI software or scripts.

6.1 Roles

Access is granted to **group roles** that describe a job. People (or service accounts) are made members of a group; nobody is granted on objects directly.

Role	Stakeholder (proposal §4.1)	Can read	Cannot read
`driw_executive`	Executive leadership	`mart.*` aggregates	warehouse detail, staging, raw, quarantine
`driw_analyst`	Data analysts	`dw.*` (all rows), `mart.*`, `secure.*`, DQ scorecards (`dq_rule_summary`, `dq_reconciliation`)	raw, staging, quarantine
`driw_regional`	Operational managers in a region	`dw.*` and `secure.*`, **only rows in their own region(s)**	marts (cross-region), other regions, the region mapping
`driw_steward`	Support staff / data engineers	`raw`, `staging`, `ref`, `meta`, all of `dq` incl. quarantine; may edit alias maps in `ref`	`dw`, `mart`

This follows **least privilege**: executives see conclusions, analysts see detail, regional officers see their own area, and stewards work on data quality without browsing the finished warehouse.

Demo personas used in tests and the presentation (NOLOGIN, used with `SET ROLE`): `driw_demo_executive`, `driw_demo_analyst`, `driw_demo_bicol`, `driw_demo_cagayan`, `driw_demo_steward`.

The web back-end's login

The PHP API (`web/php/api`, served by XAMPP) connects as `driw_web`, a LOGIN role whose only membership is `driw_executive`. A bug or an injection in the web tier can therefore reach the aggregated marts but never warehouse detail, raw data or quarantine (tested in `tests/test_php_api.py`). It also has a 10-second statement timeout and a 20-connection limit. Its password lives only in the git-ignored `.env` and in a PHP config outside the web root (`C:\xampp\driw-config\config.php`), both written by `ingestion/setup_web_role.py`.

6.2 Row-level security (regional users)

`governance.region_access` maps a role to the region(s) it may see:

role_name	region_code	region
driw_demo_bicol	0500000000	Bicol Region
driw_demo_cagayan	0200000000	Cagayan Valley

Each fact table has row-level security enabled with two policies (`dbt/macros/row_security.sql`):

```
create policy analyst_all_rows on dw.fact_damage_assessment
    for select to driw_analyst using (true);

create policy regional_own_region on dw.fact_damage_assessment
    for select to driw_regional
    using (location_key in (
        select l.location_key from dw.dim_location l
        where l.region_code in (select region_code from governance.my_regions)));
```

- **Default deny.** A regional user not yet mapped to a region sees zero rows, not everything.
- **The mapping stays private.** `governance.my_regions` is a view that shows each user only their own regions. Inside a view `current_user` is the querying user, while the view reads the mapping table with its owner's rights. A `SECURITY DEFINER` function was tried first and rejected: inside one, `current_user` is the function's owner, so no user ever matched (the governance tests caught this).
- **Secure views inherit it.** The views in `secure` (`regional_event_impact`, `regional_response_summary`) are `security_invoker`, so the same view shows Bicol to the Bicol officer, Cagayan Valley to the Cagayan officer, and everything to an analyst.
- **Marts are excluded on purpose.** Materialized views cannot carry row-level security, so regional users are not granted the marts; the secure views serve them instead.

Granting a new regional officer:

```
grant driw_regional to new_officer;
insert into governance.region_access (role_name, region_code) values ('new_officer',
'0800000000');
```

6.3 Surviving rebuilds

dbt rebuilds tables on every run, which would normally drop privileges. Grants are declared in `dbt_project.yml` (`+grants`) and row-level security is a post-hook on each fact, so both are re-applied on every build. The end-to-end tests run a second `dbt build` and check that policies and grants are still in place.

6.4 Auditing access

`governance.v_access_matrix` lists, for every role and every object, whether the role can actually read it (schema `USAGE` and object `SELECT`) and whether row-level security applies. It is the access matrix above, generated from the database itself:

```
select role_name, schema_name, count(*) filter (where can_select) as readable, count(*) as objects
from governance.v_access_matrix group by 1, 2 order by 1, 2;
```

6.5 Privacy

The governing law is the **Data Privacy Act of 2012 (Republic Act 10173)** and its implementing rules. DRIW is designed so that it holds no personal information at all.

Data classification

Class	Examples in DRIW	Handling
Public	Event names and dates, PSGC geography, organization names	No restriction
Internal (aggregated operational)	Damage costs, relief values, response times	Role-based access (Executive, Analyst)
Sensitive (aggregated)	Deaths, injured, missing, persons in evacuation centers, by city/municipality	Analysts and the region's own officers only (row-level security); executives see marts only
Personal information	Names, addresses, contact details or health data of affected people or responders	**Not collected.** Never enters the warehouse.

Privacy principles applied

- **Data minimization.** Sources are reduced to counts at the lowest level the analysis needs (city/municipality, evacuation center, request). No evacuee rosters, victim lists or patient records are

loaded; a response request carries a request id, type, timestamps and team size, never the requester's identity.

- **Aggregation and small counts.** Casualty figures are aggregated per city/municipality and event. Before any public release, counts below 5 should be suppressed or rounded to prevent re-identification in small barangays (a reporting rule for published outputs).
- **Purpose limitation.** Data is used only for disaster-response analysis and planning, as stated in the business case.
- **Least privilege and need to know.** Four roles; regional officers see only their region; the web back-end logs in as `driw_web`, which can read only aggregated marts.
- **Security of processing.** Credentials are never in source code (git-ignored `.env`, PHP config outside the web root); the API accepts only allow-listed, validated GET requests; all access is enforced inside PostgreSQL, so it holds for every client.
- **Retention.** Raw extracts are replaced on every load (full reload). Warehouse history is kept for the analysis period (2020 onwards); source files under `data/sources/` are regenerable and not retained in version control.
- **Transparency.** Every published figure is traceable to its source file and row (lineage), and every rejected row is kept with its reasons (quarantine).
- **Synthetic data.** In this implementation all operational figures are synthetic; no real person's data was processed at any stage.

If real data is onboarded: sign data-sharing agreements with each agency, register the processing with the National Privacy Commission where required, appoint a data protection officer, run a privacy impact assessment, and enable PostgreSQL connection and statement logging (`log_connections`, `pgaudit`) for an access audit trail.

6.6 Supabase

The same migrations run on Supabase. There, signed-in users connect as Supabase's `authenticated` role, so the regional mapping would key on the user's id (`auth.uid()`) or a JWT claim instead of `current_user`; the policies keep the same shape.

6.7 Standards alignment

This supports the UNDRR Data Strategy and Roadmap 2023–2027 emphasis on data governance: defined roles and responsibilities (stewardship), controlled access to sensitive operational data (casualties by locality), and auditable, reproducible data handling.

7. Analytics: OLAP and Dashboards

7.1 OLAP approach (ROLAP)

DRIW implements OLAP relationally (ROLAP) in PostgreSQL rather than with a separate MOLAP cube engine. The star schema is the cube: facts hold the measures, dimensions hold the hierarchies, and SQL's OLAP operators produce the cube's cells.

OLAP operation	How it is done	Example
Roll-up	`GROUP BY ROLLUP`, or grouping at a higher hierarchy level	Q02: damage by year with yearly subtotals
Drill-down	Group by a lower level of a hierarchy (region → province → city/municipality)	Q03
Slice	Fix one dimension value (`WHERE`)	Q07 over-capacity centers; Q09 life-saving requests
Dice	Filter on several dimensions at once	Q11 organization × resource category
Pivot	Turn dimension values into columns (`FILTER`)	Q04 cyclone vs flood per region
Cube	Every combination of dimensions with subtotals (`GROUP BY CUBE`)	Q05; `mart.mart_impact_cube`
Grouping sets	Chosen roll-ups in one pass	`mart.mart_response_performance`

Pre-aggregation. Six materialized views in the `mart` schema hold the aggregates the dashboards need. They refresh on every build and are tested to add back up to the facts. Reading a mart is about 78× faster than aggregating the facts (see query-performance.md).

The full set of sample analytical queries, each tied to a business question, follows this chapter and is in `sample-queries.sql`.

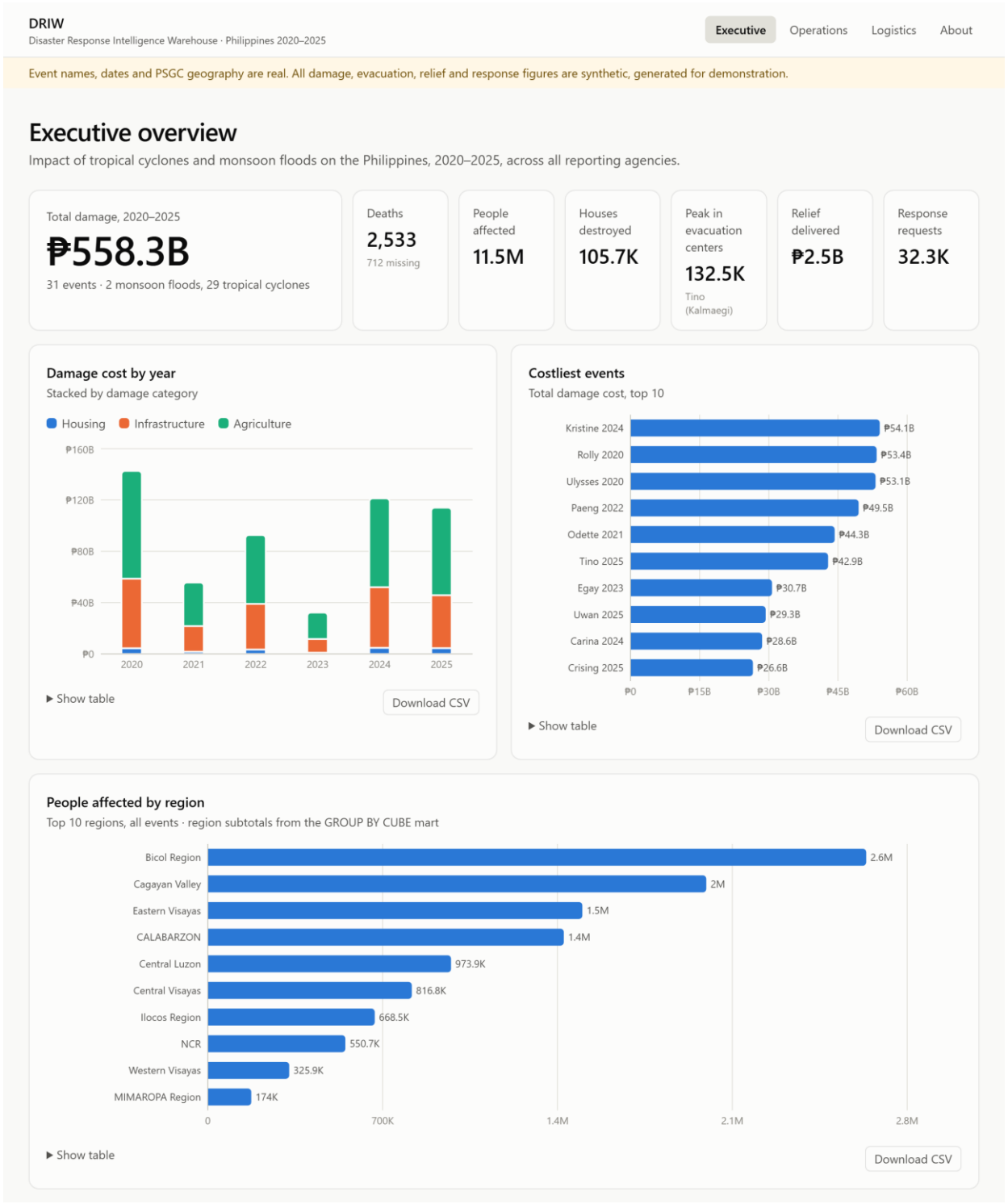
7.2 Dashboards

Built with Next.js and served by Apache (XAMPP) at `http://localhost/driw/`. A PHP API (`/driw/api/marts.php`) reads the marts live from PostgreSQL as the read-only role `driw_web`; if the warehouse is unreachable, the dashboards fall back to a bundled snapshot and say so in the footer.

Every chart has a legend, a "Show table" view with the exact numbers, and **Download CSV** of the raw values. Colors were validated for color-blind safety in light and dark themes.

Executive overview (leadership)

Total damage, casualties, people affected, peak evacuees, relief delivered; damage cost by year stacked by category; the costliest events; people affected by region (from the CUBE mart). Answers BQ1, BQ2, BQ4.


DRIW
Disaster Response Intelligence Warehouse · Philippines 2020–2025
Executive
Operations
Logistics
About
Event names, dates and PSGC geography are real. All damage, evacuation, relief and response figures are synthetic, generated for demonstration.
Executive overview
Impact of tropical cyclones and monsoon floods on the Philippines, 2020–2025, across all reporting agencies.
Total damage, 2020–2025
₱558.3B
31 events · 2 monsoon floods, 29 tropical cyclones
Deaths
2,533
712 missing
People affected
11.5M
Houses destroyed
105.7K
Peak in evacuation centers
132.5K
Tino (Kalmaegi)
Relief delivered
₱2.5B
Response requests
32.3K
Damage cost by year
Stacked by damage category
Housing
Infrastructure
Agriculture
₱160B
₱120B
₱80B
₱40B
₱0
2020
2021
2022
2023
2024
2025
▶ Show table
Download CSV
Costliest events
Total damage cost, top 10
Kristine 2024 ₱54.1B
Rolly 2020 ₱53.4B
Ulysses 2020 ₱53.1B
Paeng 2022 ₱49.5B
Odette 2021 ₱44.3B
Tino 2025 ₱42.9B
Egay 2023 ₱30.7B
Uwan 2025 ₱29.3B
Carina 2024 ₱28.6B
Crising 2025 ₱26.6B
₱0
₱15B
₱30B
₱45B
₱60B
▶ Show table
Download CSV
People affected by region
Top 10 regions, all events · region subtotals from the GROUP BY CUBE mart
Bicol Region 2.6M
Cagayan Valley 2M
Eastern Visayas 1.5M
CALABARZON 1.4M
Central Luzon 973.9K
Central Visayas 816.8K
Ilocos Region 668.5K
NCR 550.7K
Western Visayas 325.9K
MIMAROPA Region 174K
0
700K
1.4M
2.1M
2.8M
▶ Show table
Download CSV

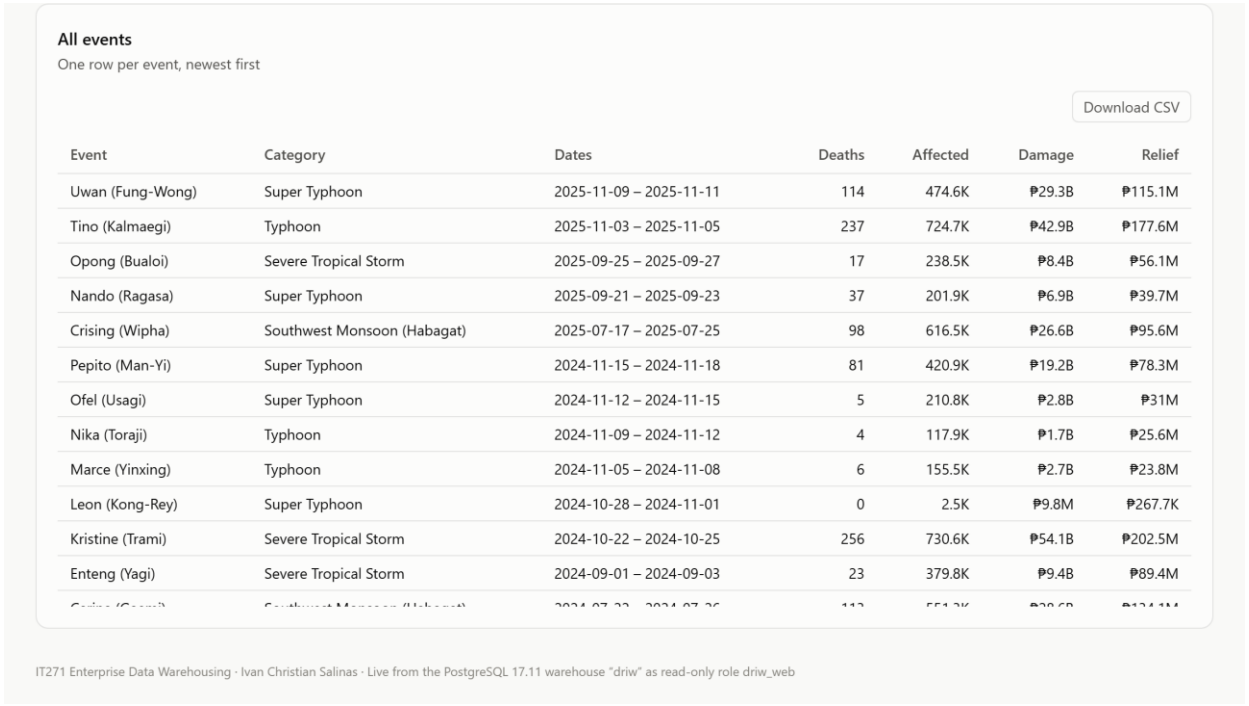


Figure: Executive dashboard

Operations (regional operations managers)

For one event (selectable, and shareable as a link such as `/driw/operations/?event=TC2024-KRISTINE`): people in evacuation centers per day, centers over capacity, and median dispatch time by response type and priority. Answers BQ5, BQ6, BQ7.

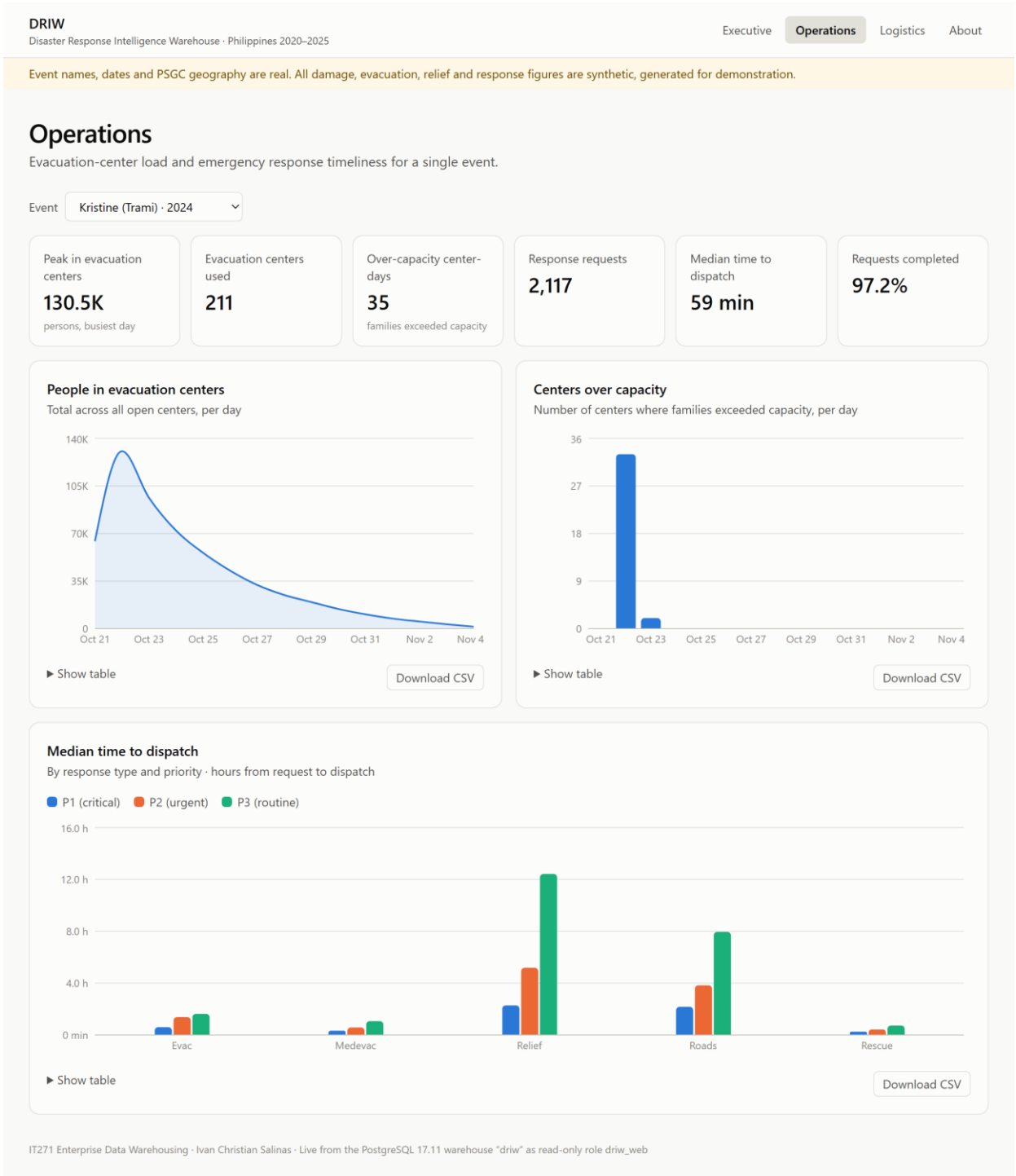


Figure: Operations dashboard

Logistics (relief and logistics officers)

Relief by organization stacked by resource category, the least-served provinces by relief per affected person, and the full coverage table. Answers BQ8, BQ9.

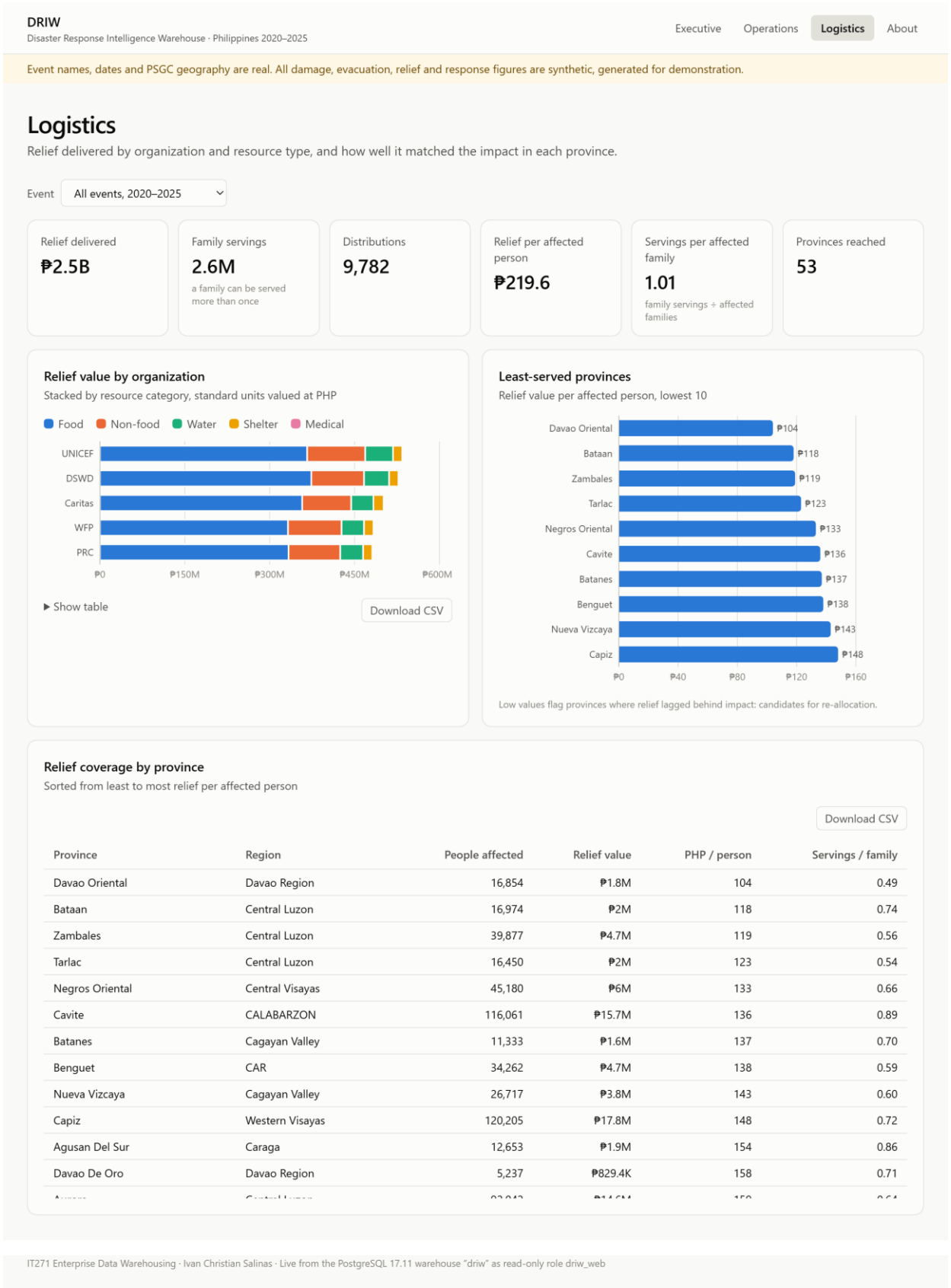


Figure: Logistics dashboard

7.3 Sample findings (synthetic data)

- **Kristine (2024)** was the costliest event at about ₱54 billion; total damage across the 31 events is ₱558.3 billion.
- **Bicol Region** had the most people affected (about 2.6 million), mainly from tropical cyclones.
- Evacuation peaks on landfall day and decays over about two weeks; overcrowding is concentrated in the first two days, which is when extra centers should open.
- **Relief delivery and road clearing** wait longest for dispatch, especially at routine priority; rescues go out fastest.
- **Davao Oriental** received the least relief per affected person (about ₱104), a candidate for re-allocation.

These figures come from synthetic operational data and illustrate the analysis, not real outcomes.

8. Testing and Validation

How DRIW was tested, and the results. Data seed 271, run on PostgreSQL 17.11.

8.1 Summary

Suite	Checks	Result
dbt build (models + data tests)	99	**99 pass**
End-to-end tests (`tests/test_end_to_end.py`)	28	**28 pass**
Governance tests (`tests/test_governance.py`)	21	**21 pass**
PHP API tests (`tests/test_php_api.py`)	13	**13 pass** (skipped where PHP is not installed, e.g. CI)
Query performance (`ingestion/benchmark_queries.py`)	14 queries	**14 within the 1 s budget**, slowest about 32 ms
Dashboards (`npm run lint`, `npm run build`)	lint + type check + static build	**pass**

Run everything with `.venv/Scripts/python -m pytest tests`. It builds a brand-new database (`driw_e2e`) from nothing through the whole pipeline (migrations, reference data, PSGC, generation, raw load, dbt, export), runs every check, then rebuilds dbt a second time. Duration: about one and a half minutes. GitHub Actions runs the same suite and the dashboard build on every push (`.github/workflows/ci.yml`). Query timings are in query-performance.md.

8.2 What is validated

1. Completeness: nothing is lost between layers

- Every raw row reaches staging (5 sources; NGO checked at item-line level).
- Every staged row has a status; every rejected row is in quarantine.
- Every valid/warning row reaches its fact table (dbt `assert_facts_reconcile_to_staging`).
- Every layer, fact and mart is non-empty.

2. Correctness: data quality is proven, not assumed

- **19/19 planted defect types detected at exactly the planted count** (`dq.dq_reconciliation`).
- Place-name matching checked against ground truth: **0 false matches** across ~63,000 rows.
- Every rejection traces to a planted defect (`warn_unplanted_rejections`).

3. Consistency: aggregates agree with detail

- Damage totals are equal in facts, the event mart and the CUBE grand total.
- Every mart adds back up to its facts (`assert_marts_reconcile_to_facts`).
- Exported JSON row counts equal mart row counts, and the dashboard data committed in `web/src/data/` is identical to a fresh build (fails if someone forgets to re-export).

4. Model integrity

- Primary and foreign keys on the star schema (9 PKs, 26 FKs).
- One row per declared grain in each fact.
- SCD Type 2: no overlapping or missing validity periods, exactly one current version; Typhoon Paeng (Oct 2022) rolls up to Maguindanao del Norte/del Sur.

5. Reproducibility

- The generator is deterministic: two runs with the same seed produce identical content.
- A fresh database built from scratch matches the working database row for row (83,442 fact rows, 1,548 Bicol damage rows in both).
- A second dbt build gives identical fact counts and keeps grants and row-level security.

6. Security (see governance.md)

- Executive reads marts only; denied on warehouse detail, staging, raw, quarantine.
- Analyst sees every fact row, marts and DQ scorecards; denied raw and quarantine.
- Regional officers (Bicol, Cagayan Valley) see exactly their region's rows in all four facts and in the secure views, and zero rows from other regions.
- An unmapped regional user sees zero rows (default deny).
- Regional users cannot read the mapping, grant themselves roles, or edit their mapping.
- Stewards work on raw, staging and quarantine, may fix alias maps, and are denied the warehouse.

8.3 Defects found by testing (and fixed)

Testing earned its keep. Each of these was caught by a test, not by inspection:

Found by	Problem	Fix
DQ reconciliation	Generator counted the placeholder "-" as a negative number	Count only real numbers as negative
Unplanted-rejection check	Monsoon incidents named "Southwest Monsoon (Habagat) Carina (Gaemi)" matched the first parentheses; 1,244 rows wrongly rejected	Match the last parentheses
Location accuracy test	Letter swaps inside "City of" (e.g. "Cityo f Legazpi") went unmatched	Letters-only fuzzy key
Governance tests	Regional users saw **zero** rows: a `SECURITY DEFINER` function sees its owner as `current_user`	Region lookup moved into a view
Governance tests	`SET ROLE` escalation test passed for the wrong reason (checked against the session user)	Test escalation via `GRANT` and mapping writes instead
Browser review	"Family coverage 101%" double-counted repeat distributions	Relabeled "servings per affected family"

8.4 Known limitations

- All operational figures are synthetic; magnitudes are calibrated to look plausible, not to match official NDRRMC totals.
- City population is estimated from barangay counts (the PSA workbook with census population could not be downloaded automatically).
- The PSGC source predates the 2022 Maguindanao split; the split is applied through the SCD2 change log.
- Seven misspelled place names are equally close to two real towns and stay unmatched by design, for a data steward to resolve.
- Dashboards read a JSON snapshot of the marts; live querying needs the warehouse on Supabase (one file, `web/src/lib/data.ts`).

8.5 Query Performance Test Report

Generated by `ingestion/benchmark_queries.py` on 2026-10-16 12:41.

Method

- Every query in `sample-queries.sql` runs once to warm the cache, then 5 times under `EXPLAIN (ANALYZE)`. Times are server-side planning + execution, in milliseconds.
- Acceptance criterion (NFR2, business case): every analytical query completes in under 1000 ms.
- Environment: PostgreSQL 17.11 on Windows 11 (AMD64), local laptop.
- Data volume: fact_damage_assessment 8,806 rows, fact_evacuation_snapshot 31,110 rows, fact_resource_distribution 11,179 rows, fact_response_action 32,347 rows.

Results

Query	Business question	OLAP operation	Rows	Median ms	Slowest ms	Within budget
Q01	Costliest events and their human toll (BQ1)	Ranking (top-N)	10	3.60	3.73	Yes
Q02	Damage by year and category, with yearly subtotals (BQ2)	Roll-up (GROUP BY ROLLUP)	31	4.57	4.74	Yes
Q03	Drill-down: region -> province -> city/municipality for one event (BQ3)	Drill-down	15	1.21	1.28	Yes
Q04	Tropical cyclone vs monsoon flood impact per region (BQ4)	Pivot (FILTER) / dice	15	4.08	4.70	Yes
Q05	Every subtotal of impact by year x region x hazard (BQ2, BQ4)	Cube (GROUP BY CUBE)	160	11.48	11.63	Yes
Q06	Peak evacuees per event (semi-additive measure) (BQ5)	Aggregate over a snapshot	10	4.74	5.13	Yes
Q07	Evacuation centers most often over capacity (BQ6)	Slice (is_over_capacity)	10	17.54	18.16	Yes
Q08	Dispatch time by response type and priority (BQ7)	Accumulating snapshot analysis	15	30.84	31.27	Yes
Q09	Slowest-responding cities/municipalities for life-saving requests (BQ7)	Slice + ranking	10	31.98	32.94	Yes
Q10	Least relief per affected person, by province (BQ8)	Integration of two facts	10	6.91	8.30	Yes
Q11	Relief mix by organization and resource category (BQ9)	Dice (org x category)	25	5.82	5.93	Yes
Q12	Year-over-year change in damage cost (BQ2)	Window function (LAG)	6	2.44	2.61	Yes
Q13	Dashboard query served from a mart (BQ1, BQ5, BQ8)	Pre-aggregated ROLAP mart	10	0.05	0.05	Yes
Q14	What data was rejected, and why? (BQ10)	Data-quality audit	12	0.23	0.24	Yes

Findings

- **14 of 14 queries** meet the 1000 ms budget on every run; the slowest single run was Q09 at 32.9 ms.
- **Marts pay off:** the event headline read from `mart.mart_event_summary` (Q13, 0.05 ms) is about 75× faster than aggregating the same figures from the fact table (Q01, 3.60 ms). This is why the dashboards read marts, not facts.
- **Sequential scans are the right plan at this size.** With tens of thousands of rows per fact, PostgreSQL reads each table in a few milliseconds and chooses sequential scans over indexes. At national scale (millions of rows) the next step is B-tree indexes on the fact foreign keys most used in filters (`event_key`, `location_key`, date keys) and partitioning facts by event year; this benchmark is the baseline to measure that against.
- Plans per query (scan types used):
- Q01: Seq Scan
- Q02: Seq Scan
- Q03: Seq Scan
- Q04: Seq Scan
- Q05: Seq Scan
- Q06: Seq Scan
- Q07: Seq Scan
- Q08: Seq Scan

- Q09: Seq Scan
- Q10: Seq Scan
- Q11: Seq Scan
- Q12: Seq Scan
- Q13: Seq Scan
- Q14: Seq Scan

9. Implementation and Live Demo

9.1 Technology stack

Layer	Technology
Data warehouse	PostgreSQL 17.11 (local)
ETL	Python 3.13 (extraction, loading), dbt Core 1.12 (transformation, data quality, tests)
OLAP	ROLAP in PostgreSQL: star schema, materialized views, `CUBE` / `ROLLUP` / `GROUPING SETS`
Back-end	PHP 8.1 on XAMPP (Apache) with `pdo_pgsql`
Dashboards	Next.js (static build) served by Apache
Version control and CI	GitHub (`sardinexszc/NEUSTIT271_EDW_Project`), GitHub Actions

9.2 Repository layout

Folder	Contents
`ingestion/`	Python pipeline: migrations, reference loads, synthetic generator, raw loader, export, benchmark, web role
`supabase/migrations/`	SQL schema history (schemas, raw tables, reference tables, roles, governance)
`dbt/`	Staging, data-quality, warehouse and mart models; tests; rule catalog
`data/reference/`	Real reference data (events, PSGC-derived geography, organizations, relief items)
`web/`	Dashboards (Next.js) and the PHP API (`web/php/api`)
`tests/`	End-to-end, governance and API tests (pytest)
`docs/`	This documentation, diagrams and their generators

9.3 Build and run

```
# 1. Python environment and database connection (.env holds PG* settings)
python -m venv .venv
.venv/Scripts/pip install -r requirements.txt

# 2. Build the warehouse end to end (about 1 minute)
.venv/Scripts/python ingestion/pipeline.py

# 3. Validate
.venv/Scripts/python -m pytest tests

# 4. Web back-end login + PHP config (once), then publish to XAMPP
.venv/Scripts/python ingestion/setup_web_role.py
cd web && npm install && npm run deploy:xampp
```

XAMPP's `php.ini` needs `extension=pdo_pgsql` and `extension=pgsql` enabled, and Apache restarted.

9.4 Live demo script

1. **Warehouse.** In pgAdmin or psql, show the layers (`raw` → `staging` → `dw` → `mart`) and the ER diagram of a star (pgAdmin draws it from the real PK/FK constraints).
2. **Data quality.** `select * from dq.dq_reconciliation;` shows all 19 planted defect types detected at the exact count; `select * from dq.dq_quarantine limit 10;` shows rejected rows with reasons.
3. **OLAP.** Run Q02 (roll-up), Q03 (drill-down), Q05 (cube) and Q06 (semi-additive peak) from `docs/sample-queries.sql`.

4. **SCD Type 2.** `select * from dw.dim_location where city_muni_name = 'Parang' and psgc_code like '19038%';` shows Maguindanao before and Maguindanao del Norte after 18 Sep 2022.
5. **Security.** `begin; set local role driw_demo_bicol; select count(*) from dw.fact_damage_assessment; rollback;` returns only Bicol's rows; as `driw_demo_executive` the same query is refused.
6. **Dashboards.** Open `http://localhost/driw/`: Executive, then Operations for Kristine (`/driw/operations/?event=TC2024-KRISTINE`), then Logistics. Point out the footer ("Live from the PostgreSQL warehouse … as read-only role driw_web") and Download CSV.
7. **API.** `http://localhost/driw/api/health.php` and `http://localhost/driw/api/marts.php?name=mart_event_summary`.

Appendix A. Sample Analytical Queries

Each query answers a key business question (section 1.3) and demonstrates an OLAP operation. Timings are in the Query Performance Test Report (Chapter 8). Source: docs/sample-queries.sql.

Q01. Costliest events and their human toll (BQ1)

OLAP operation: Ranking (top-N)

```
select e.event_name,
       e.category,
       sum(f.cost_php)          as damage_cost_php,
       sum(f.dead)              as dead,
       sum(f.affected_persons)  as affected_persons
from dw.fact_damage_assessment f
join dw.dim_event e using (event_key)
group by e.event_name, e.category
order by damage_cost_php desc
limit 10;
```

Q02. Damage by year and category, with yearly subtotals (BQ2)

OLAP operation: Roll-up (GROUP BY ROLLUP)

```
select coalesce(e.event_year::text, 'All years')            as event_year,
       coalesce(dc.damage_category, 'All categories')       as damage_category,
       sum(f.cost_php)                                      as damage_cost_php
from dw.fact_damage_assessment f
join dw.dim_event e using (event_key)
join dw.dim_damage_category dc using (damage_category_key)
group by rollup (e.event_year, dc.damage_category)
order by e.event_year nulls last, dc.damage_category nulls last;
```

Q03. Drill-down: region -> province -> city/municipality for one event (BQ3)

OLAP operation: Drill-down

```
select l.region_name,
       l.province_name,
       l.city_muni_name,
       sum(f.affected_persons) as affected_persons,
       sum(f.cost_php)         as damage_cost_php
from dw.fact_damage_assessment f
join dw.dim_location l using (location_key)
join dw.dim_event e using (event_key)
where e.event_code = 'TC2024-KRISTINE'
group by l.region_name, l.province_name, l.city_muni_name
order by affected_persons desc
limit 15;
```

Q04. Tropical cyclone vs monsoon flood impact per region (BQ4)

OLAP operation: Pivot (FILTER) / dice

```
select l.region_name,
       sum(f.affected_persons) filter (where e.hazard_type = 'Tropical Cyclone')
           as cyclone_affected,
       sum(f.affected_persons) filter (where e.hazard_type = 'Flood')
```

```
           as flood_affected,
       sum(f.affected_persons) as total_affected
from dw.fact_damage_assessment f
join dw.dim_event e using (event_key)
join dw.dim_location l using (location_key)
group by l.region_name
order by total_affected desc;
```

Q05. Every subtotal of impact by year x region x hazard (BQ2, BQ4)

OLAP operation: Cube (GROUP BY CUBE)

```
select e.event_year, l.region_name, e.hazard_type,
       grouping(e.event_year, l.region_name, e.hazard_type) as grouping_id,
       sum(f.dead)            as dead,
       sum(f.affected_persons) as affected_persons,
       sum(f.cost_php)        as damage_cost_php
from dw.fact_damage_assessment f
join dw.dim_event e using (event_key)
join dw.dim_location l using (location_key)
group by cube (e.event_year, l.region_name, e.hazard_type);
```

Q06. Peak evacuees per event (semi-additive measure) (BQ5)

OLAP operation: Aggregate over a snapshot

```
select e.event_name,
       max(daily.persons) as peak_persons_in_centers,
       count(*)           as days_with_evacuees
from (
    -- sum across centers per day
    select event_key, snapshot_date_key, sum(persons_inside) as persons
    from dw.fact_evacuation_snapshot
    group by event_key, snapshot_date_key
) daily
join dw.dim_event e using (event_key)
group by e.event_name   -- then MAX over days, never SUM
order by peak_persons_in_centers desc
limit 10;
```

Q07. Evacuation centers most often over capacity (BQ6)

OLAP operation: Slice (is_over_capacity)

```
select c.ec_name, c.ec_type, l.province_name,
       count(*) filter (where f.is_over_capacity) as over_capacity_days,
       max(f.families_inside)                     as max_families,
       c.capacity_families
from dw.fact_evacuation_snapshot f
join dw.dim_evacuation_center c using (evacuation_center_key)
join dw.dim_location l using (location_key)
group by c.ec_name, c.ec_type, l.province_name, c.capacity_families
having count(*) filter (where f.is_over_capacity) > 0
order by over_capacity_days desc, max_families desc
limit 10;
```

Q08. Dispatch time by response type and priority (BQ7)

OLAP operation: Accumulating snapshot analysis

```
select rt.response_type_name,
       f.priority,
       count(*) as requests,
       round(percentile_cont(0.5) within group (order by f.hours_to_dispatch)::numeric, 2)
           as median_hours_to_dispatch,
       round(percentile_cont(0.9) within group (order by f.hours_to_dispatch)::numeric, 2)
           as p90_hours_to_dispatch,
       round(100.0 * avg(f.is_completed::int), 1) as pct_completed
from dw.fact_response_action f
join dw.dim_response_type rt using (response_type_key)
group by rt.response_type_name, f.priority
order by rt.response_type_name, f.priority;
```

Q09. Slowest-responding cities/municipalities for life-saving requests (BQ7)

OLAP operation: Slice + ranking

```
select l.province_name, l.city_muni_name,
       count(*) as life_saving_requests,
       round(percentile_cont(0.5) within group (order by f.hours_to_dispatch)::numeric, 2)
           as median_hours_to_dispatch
from dw.fact_response_action f
join dw.dim_response_type rt using (response_type_key)
join dw.dim_location l using (location_key)
where rt.response_cluster = 'Life-saving'
group by l.province_name, l.city_muni_name
having count(*) >= 20
order by median_hours_to_dispatch desc
limit 10;
```

Q10. Least relief per affected person, by province (BQ8)

OLAP operation: Integration of two facts

```
with impact as (
    select l.province_name, sum(f.affected_persons) as affected
    from dw.fact_damage_assessment f join dw.dim_location l using (location_key)
    group by l.province_name
), relief as (
    select l.province_name, sum(f.value_php) as relief_php
    from dw.fact_resource_distribution f join dw.dim_location l using (location_key)
    group by l.province_name
)
select i.province_name, i.affected, coalesce(r.relief_php, 0) as relief_php,
       round(coalesce(r.relief_php, 0) / nullif(i.affected, 0), 2) as relief_php_per_person
from impact i left join relief r using (province_name)
order by relief_php_per_person
limit 10;
```

Q11. Relief mix by organization and resource category (BQ9)

OLAP operation: Dice (org x category)

```
select o.org_name, r.category,
       sum(f.quantity)  as quantity_standard_units,
       sum(f.value_php) as relief_value_php
```

```
from dw.fact_resource_distribution f
join dw.dim_organization o using (organization_key)
join dw.dim_resource r using (resource_key)
group by o.org_name, r.category
order by o.org_name, relief_value_php desc;
```

Q12. Year-over-year change in damage cost (BQ2)

OLAP operation: Window function (LAG)

```
select event_year,
       damage_cost_php,
       round(100.0 * (damage_cost_php - lag(damage_cost_php) over (order by event_year))
             / nullif(lag(damage_cost_php) over (order by event_year), 0), 1)
           as pct_change_vs_prior_year
from (
    select e.event_year, sum(f.cost_php) as damage_cost_php
    from dw.fact_damage_assessment f join dw.dim_event e using (event_key)
    group by e.event_year
) yearly
order by event_year;
```

Q13. Dashboard query served from a mart (BQ1, BQ5, BQ8)

OLAP operation: Pre-aggregated ROLAP mart

```
select event_name, damage_cost_php, peak_persons_in_centers, relief_php_per_affected_person
from mart.mart_event_summary
order by damage_cost_php desc
limit 10;
```

Q14. What data was rejected, and why? (BQ10)

OLAP operation: Data-quality audit

```
select source_system, reject_reasons, count(*) as rows_quarantined
from dq.dq_quarantine
group by source_system, reject_reasons
order by rows_quarantined desc;
```